AN-NAJAH NATIONAL UNIVERSITY
Faculty of Engineering & Information Technology
Computer Engineering Department

Graduation Project

Healthcare Ecosystem

Authors
Abdulrhman Sawalmeh

Supervisor
Dr. Shareef Yaseen

Date
September 2026

Nablus, Palestine

Acknowledgements

The authors would like to express their sincere gratitude to their supervisor, Dr. Shareef Yaseen, for his guidance and support throughout the project. We also thank the Computer Engineering Department, our families, colleagues, and everyone who provided feedback and encouragement during the development of Healthcare Ecosystem.

Dedication

Dedicated to our families and teachers, whose support and encouragement made this work possible.

Abstract

Healthcare organizations often use separate records and communication channels, making it difficult to coordinate referrals, follow-up, and access to patient information across centers with different capabilities. Healthcare Ecosystem addresses this problem through an integrated platform that supports network administration while preserving the daily workflow of each healthcare center.

The solution consists of four major parts: a Central System for network administration and shared data; a Medium Center System for comprehensive clinical and administrative workflows; a Small Center System with a reduced workflow suited to fewer services; and a Medium-Center Mobile Application for staff and patient access. The platform manages patients, doctors, appointments, visits, referrals, laboratory requests, prescriptions and pharmacy activities, notifications, messaging, audit records, and role-based access. It also supports coordination between local center records and unified network information.

The web interfaces were developed with React, while the mobile application uses React Native and Expo. Node.js, Express, and TypeScript implement the application services, with PostgreSQL and Prisma providing relational data storage and access. Authentication, workspace scoping, and role checks separate the responsibilities of administrators, managers, clinical staff, and patients.

The completed project demonstrates how a multi-application architecture can support the main information and coordination needs of a healthcare network. It is presented as an academic prototype that can be extended through further testing, interoperability work, and deployment hardening.

Keywords: digital health; electronic health record; health information exchange; healthcare referral; role-based access control; React; Node.js; PostgreSQL; Prisma; Expo.

Table of Contents

List of Figures

List of Tables

List of Abbreviations

Abbreviation	Meaning
API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
DBMS	Database Management System
EHR	Electronic Health Record
FHIR	Fast Healthcare Interoperability Resources
HIE	Health Information Exchange
HTTP	Hypertext Transfer Protocol
JWT	JSON Web Token
MRN	Medical Record Number
ORM	Object–Relational Mapping
QR	Quick Response
RBAC	Role-Based Access Control
REST	Representational State Transfer
SMTP	Simple Mail Transfer Protocol
UI / UX	User Interface / User Experience

Chapter 1: Introduction

1.1 Background and Motivation

Digital health systems increasingly mediate the movement of clinical and administrative information among patients, clinicians, laboratories, pharmacies, and healthcare organizations. The World Health Organization emphasizes that digital-health initiatives must be connected to organizational, human, and technical resources rather than treated as isolated applications [1]. In practical care networks, however, each facility may retain its own workflow, staff, capacity, and local record while patients move between providers. This creates a need for coordination without removing local operational autonomy.

Healthcare Ecosystem addresses this need as a multi-application academic prototype. A central administrative system maintains network-level center, referral, master-data, notification, analytics, reporting, and audit views. Medium and small centers operate their own role-specific portals. A mobile application exposes a subset of medium-center functions to clinical and patient roles. The project therefore studies not only record entry, but also how responsibilities and state transitions cross system boundaries.

1.2 Problem Statement

When healthcare centers use separate records and informal coordination channels, staff can encounter duplicated registration, incomplete longitudinal context, inconsistent referral status, delayed result delivery, and limited network-level oversight. The problem is especially difficult when facilities have different capabilities: a smaller center may need to refer a patient to a larger center with a suitable doctor, medicine, operating-room window, or laboratory capability. A useful platform must preserve center-level workflows while creating a controlled, auditable path for exchange.

Project problem: Design and implement a role-aware software ecosystem that coordinates healthcare-center operations and inter-center referrals while maintaining explicit authorization, traceability, and a longitudinal representation of patient activity.

1.3 Aim and Objectives

The aim is to design and implement an integrated healthcare information system that supports coordinated work across a network of healthcare centers. The main objectives are:

Table 1.1. Project objectives

ID	Objective
O1	Provide central visibility of participating centers, network activity, master data, referrals, notifications, reports, and audits.
O2	Support differentiated medium- and small-center workflows according to the capabilities implemented for each center type.
O3	Enforce role-aware access for managers, doctors, receptionists, nurses, laboratory staff, pharmacists, patients, and the central administrator.

ID	Objective
O4	Model patient registration, appointments, visits, queues, results, prescriptions, messaging, notifications, and longitudinal history.
O5	Coordinate inter-center referral requests, receiving-manager decisions, doctor assignment, visit creation, completion, cancellation, and return reasons.
O6	Expose mobile access for medium-center staff and patients through the same API family.
O7	Apply practical safeguards including authentication, role/workspace authorization, validation, password hashing, audit logs, and configurable CORS/Helmet middleware.

1.4 Development Questions

1. How can one software repository represent network-level administration together with different local-center capabilities?
2. How can role and workspace boundaries be carried consistently across web, mobile, API, and data layers?
3. How can referral, visit, result, prescription, message, and notification states be made explicit enough to support traceability?
4. How can the system remain maintainable while supporting different center capabilities and user roles?

1.5 Scope

The project includes a central administration system, medium- and small-center web systems, and a mobile application connected to the medium-center services. Its scope covers identity and role management, patient and doctor records, appointments, visits, referrals, laboratory and pharmacy activities, messaging, notifications, audit records, and selected patient self-service functions. Production infrastructure, medical-device connectivity, insurance settlement, national identity integration, and regulatory certification are outside the current project scope.

1.6 Constraints and Assumptions

- Development and testing were carried out in a local environment rather than a production healthcare network.
- The mobile application was developed with Expo; final behavior on a wider range of native devices still requires additional testing.
- The prototype care assistant depends on configured external providers or a local fallback and must not be interpreted as a validated medical diagnostic service.
- The current system uses demonstration data and has not been clinically validated for use with real patient care.

1.7 Report Structure

Chapter 2 reviews relevant literature and standards. Chapter 3 presents the development methodology and requirements. Chapter 4 explains the architecture, data design, and main workflows, followed by the implementation in Chapter 5. Chapters 6 and 7 cover testing, results, strengths, and limitations. Chapter 8 concludes the report and outlines future work. Appendix B provides a short technical summary.

Chapter 2: Literature Review and Background

2.1 Electronic Health Records and Health Information Exchange

An electronic health record organizes patient information in a digital form that can support clinical documentation, computerized ordering, decision support, and exchange. Reported benefits include improvements in information availability, quality, organizational processes, and research capacity, while drawbacks include acquisition and maintenance cost, temporary productivity disruption, workflow change, and privacy concerns [5]. These trade-offs are directly relevant to Healthcare Ecosystem: adding more connected functions increases value only if the platform also manages authorization, workflow fit, reliability, and data governance.

Health information exchange focuses on moving health information across organizational boundaries. A systematic review by Rudin et al. found evidence of value in some emergency-department contexts but also identified low usage, sustainability concerns, technical and workflow barriers, cost, and privacy issues [6]. This supports a cautious interpretation of the present project. Its contribution is a working academic workflow and integration foundation, not proof of clinical outcome improvement.

2.2 Interoperability and HL7 FHIR

HL7 FHIR defines resources and common exchange patterns for electronically shared healthcare information [2]. Healthcare Ecosystem currently exchanges project-specific JSON structures over REST routes; it does not implement FHIR resources, terminology binding, conformance profiles, or a FHIR server. FHIR is therefore a relevant future interoperability target rather than a current feature. A migration path could map patients, practitioners, organizations, appointments, encounters, service requests, observations, medication requests, communications, consents, provenance, and audit events to validated profiles.

2.3 Role-Based Access Control

Role-based access control mediates operations through organizational roles instead of individual identities. NIST's revised RBAC model formalizes users, roles, permissions, role activation, and authorization constraints [3]. The project uses a pragmatic RBAC variant: the JWT carries identity and role; API middleware verifies authentication and allowed roles; application queries also scope data to the current center or patient. The result aligns with core role-to-permission reasoning, although it does not implement a configurable permission engine, formal role hierarchy, separation-of-duty policy, or administrative role review.

2.4 API Security and Healthcare Data

Healthcare APIs combine identifiers with sensitive records, making object- and function-level authorization essential. OWASP's API Security Top 10 identifies broken object-level authorization, broken authentication, and broken function-level authorization among leading API risks [4]. The repository addresses part of this risk through server-side authentication, role checks, center scoping, patient scoping, validation, and audit logs. However, a production assessment would also require systematic negative-authorization tests, rate limiting, stronger secret management, secure deployment, dependency monitoring, encrypted backups, incident response, and privacy-impact analysis.

2.5 Related Academic Project Reports

Recent software graduation reports in An-Najah's institutional collection use formal front matter, numbered chapters, implementation figures, discussion, conclusions, and references. UniServe documents an integrated mobile and web workflow for community-service administration [12], while MARS presents a role-aware full-stack retail platform and distinguishes its implemented prototype from future production work [13]. These reports provide useful examples of academic structure and presentation.

2.6 Identified Gap and Project Position

The reviewed background indicates that connected health platforms require more than a single EHR screen. They must coordinate organizational boundaries, roles, longitudinal data, operational states, security, and interoperability. Healthcare Ecosystem addresses this need through central network administration, two local-center capability profiles, inter-center referral states, local visit workflows, patient-facing functions, and mobile access within one TypeScript project.

Table 2.1. Background concepts and their project implications

Concept	Established concern	Project response	Remaining gap
EHR	Complete, usable clinical context	Patient, visit, result, prescription, and timeline models	No production clinical validation or certified terminology
HIE	Cross-organization exchange and sustainability	Central/local referral and notification queues	No national exchange or delivery guarantee
FHIR	Standardized resources and conformance	REST/JSON foundation	No FHIR implementation
RBAC	Roles and authorized transactions	JWT role and workspace middleware	No configurable policy engine or formal SoD
API security	Object/function authorization and safe configuration	Scoping, validation, Helmet, CORS, auditing	Requires automated security testing and deployment hardening

Chapter 3: Development Methodology and Requirements

3.1 Development Approach

Healthcare Ecosystem was developed iteratively. Each cycle focused on a group of users or a complete workflow, followed by integration with the relevant API and database models. This approach allowed the team to refine requirements as the relationships between the central system, local centers, and mobile application became clearer.

5. Identify the healthcare actors and the information needed by each role.
6. Define role permissions and the responsibilities of central, medium, and small centers.
7. Design the PostgreSQL data models and Prisma relationships for shared and local information.
8. Implement the Express APIs and their authentication, validation, and workflow rules.
9. Build the React web interfaces and the React Native/Expo mobile interface.
10. Integrate referral, notification, visit, laboratory, pharmacy, and patient workflows.
11. Test the main functions, correct issues, and refine the interfaces.

3.2 Requirements Analysis

Requirements were organized around eight actors: the central administrator, center manager, doctor, receptionist, nurse, laboratory staff, pharmacist, and patient. Their responsibilities were used to define the available pages, API permissions, and data access boundaries.

Table 3.1. Role availability by user-facing system

Role	Central web	Medium web	Small web	Mobile
Central administrator	Yes	No	No	No
Center manager	No	Yes	Yes	Yes
Doctor	No	Yes	Yes	Yes
Receptionist	No	Yes	Yes	Yes
Nurse	No	Yes	No	Yes
Laboratory staff	No	Yes	No	Yes
Pharmacist	No	Yes	No	Yes
Patient	No	Yes	Yes	Yes

Table 3.2. Functional requirements

ID	Requirement	Primary scope
FR-01	Authenticate users and support password recovery.	All clients
FR-02	Apply role- and workspace-based navigation and API authorization.	All systems
FR-03	Manage healthcare centers and central reference data.	Central
FR-04	Register, search, view, update, and QR-identify patients.	Central and local centers
FR-05	Manage doctors and their availability.	Local centers

ID	Requirement	Primary scope
FR-06	Create and coordinate inter-center referrals.	Central and local centers
FR-07	Create visits and progress center-specific queue states.	Medium and small centers
FR-08	Manage laboratory requests and results.	Medium center
FR-09	Manage prescriptions, dispensing, and pharmacy inventory.	Medium center
FR-10	Book and manage appointments.	Patient and doctor portals
FR-11	Support patient–doctor messages and read status.	Local web and mobile
FR-12	Process notifications between centers and the central system.	All APIs
FR-13	Record auditable administrative and clinical events.	Authorized roles
FR-14	Provide patient permissions and a guarded prototype care assistant.	Selected local and mobile flows

The main non-functional requirements are:

- Security through authenticated APIs, role checks, center and patient scoping, password hashing, input validation, and audit records.
- Usability through role-specific navigation, searchable lists, clear status indicators, responsive layouts, and understandable error feedback.
- Maintainability through TypeScript, modular services and interfaces, declarative Prisma schemas, and versioned migrations.
- Reliability through explicit workflow states, conflict checks, notification attempts, and consistent server-side error handling.
- Portability through browser-based clients and an Expo mobile application connected through REST APIs.
- Privacy by limiting access to the information required by each role and center.

3.3 System Design

The system was divided into central, medium-center, small-center, and mobile parts. Use cases were mapped to the responsible actors before the APIs and database relationships were finalized. Shared concepts such as identity, referrals, and notifications connect the applications, while local clinical records remain associated with the center that created them.

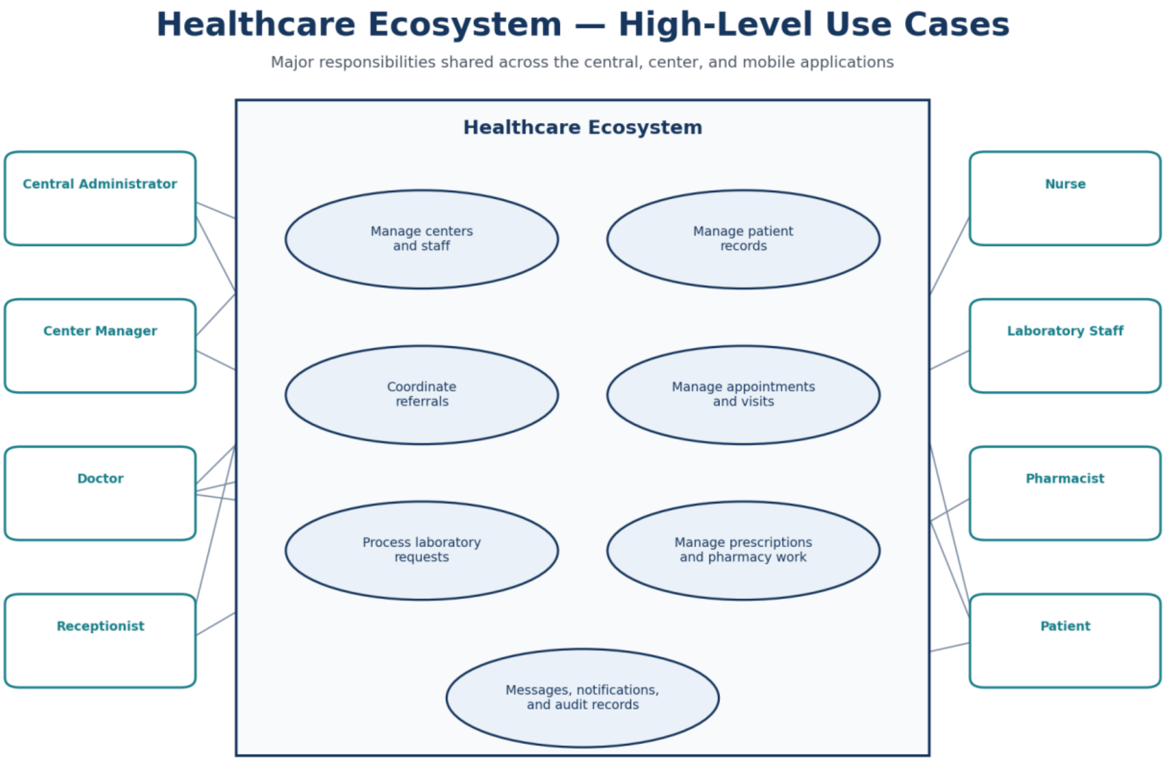

Figure 3.1. High-level use-case diagram for Healthcare Ecosystem.

3.4 Implementation

Implementation proceeded from the data and API layers to the user interfaces. Prisma models represented patients, staff, visits, referrals, appointments, laboratory requests, prescriptions, messages, notifications, and audit records. Express routes applied business rules and authorization, while the React and Expo clients presented the functions available to each role.

3.5 Integration

Integration focused on workflows that cross application boundaries. Referral requests connect local centers with central coordination; notification records communicate changes between systems; and the mobile application uses the medium-center API. Shared TypeScript definitions and consistent authentication patterns reduce differences between these parts.

3.6 Testing and Validation

The team validated the project through workspace builds, API health checks, role-based login, page and workflow checks, and review of database-backed behavior. Tests concentrated on the main user journeys and on confirming that each role reached the correct workspace. Chapter 6 summarizes the functional results and clearly separates fully exercised scenarios from those requiring a final database-alignment retest.

Chapter 4: System Design and Architecture

4.1 Monorepository Architecture

The project uses NPM workspaces to separate three APIs, three web clients, one mobile client, and a shared TypeScript package. This structure allows each center type to implement its own workflow while keeping common development conventions and domain definitions.

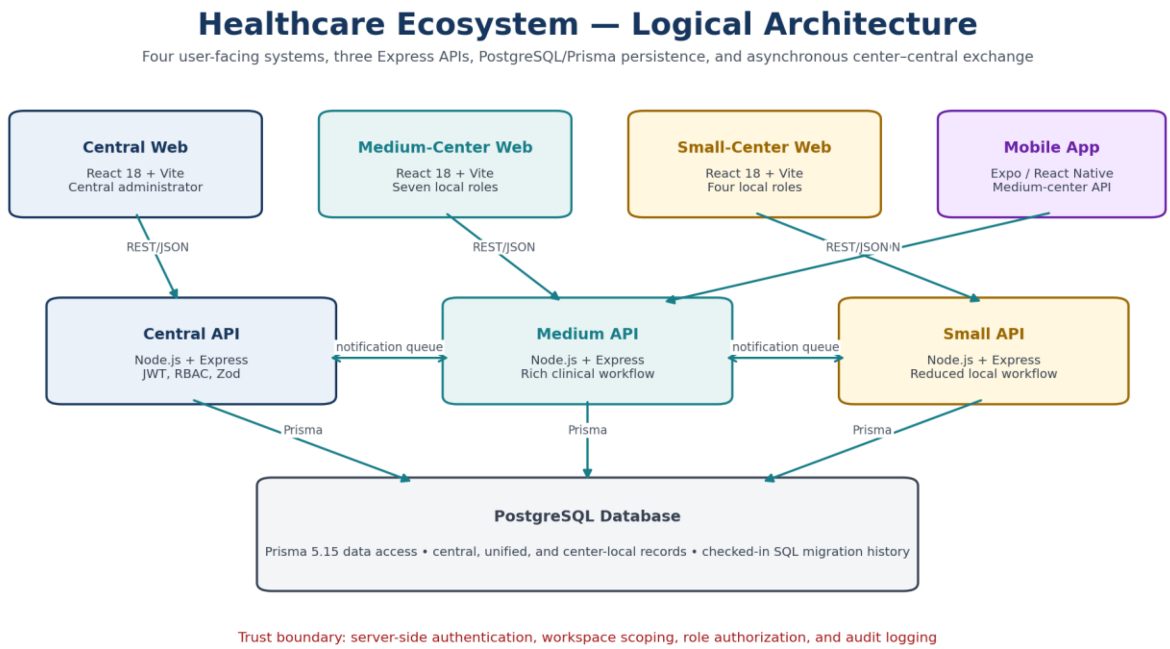

Figure 4.1. Logical architecture of the Healthcare Ecosystem.

Table 4.1. Application packages and responsibilities

Package	Layer	Responsibility
@healthcare/central-api	Express API	Network administration, central records, exchange processing, and authentication
@healthcare/central-web	React web	Central administrator portal, analytics, reports, and audits
@healthcare/medium-center-api	Express API	Full local clinical workflow, patient portal, laboratory, pharmacy, consent, and referral functions
@healthcare/medium-center-web	React web	Seven-role medium-center portal
@healthcare/small-center-api	Express API	Reduced local visit/referral/patient portal functions
@healthcare/small-center-web	React web	Four-role small-center portal
@healthcare/medium-center-mobile	Expo / React Native	Medium-center staff and patient mobile experience
@healthcare/shared	TypeScript	Shared domain declarations

4.2 Technology Stack

Table 4.2. Implementation technologies

Area	Technology/version	Role in project
Web UI	React 18.3, React Router 6.28, Vite 5.4	Component-based role portals and client routing
Mobile UI	React 19.1, React Native 0.81.5, Expo 54	Cross-platform mobile client
Language	TypeScript 5.6	Typed clients, APIs, schemas, and shared definitions
Server	Node.js, Express 4.19	REST/JSON endpoints and background processors
Data	PostgreSQL, Prisma 5.15	Relational persistence, schema models, and migrations
Authentication	jsonwebtoken 9, bcryptjs 2.4	JWT access tokens and password/code hashing
Validation/security	Zod 3.23, Helmet 7.1, CORS	Request validation and HTTP middleware
Reporting/visuals	PDFKit 0.19, Recharts 3.8	Medium API PDF output and central charts

4.3 Data Architecture

The data architecture separates network information from center-specific clinical work. Central records describe participating centers, CenterUserAccount identities, UnifiedPatient and unified visit information, CentralReferral coordination, shared catalogues, availability, notification queues, and AuditLog entries. Each center keeps LocalPatient and LocalVisit records for its own workflow. Medium-center records also connect visits to LabRequestLocal, LocalPrescription, nursing assessments, result reports, inventory, and related clinical activities.

A local patient is linked to the corresponding unified identity used across the network, allowing a center to retain its operational record while the central system coordinates referrals and selected shared information. IncomingNotification and OutgoingNotification records carry changes between local and central services. Doctor profiles are associated with center accounts, and prescriptions and laboratory requests remain connected to the local visit in which they were created.

Table 4.3. Prisma schema size

Application	Models	Enums	Primary purpose
central-api	45	22	Network coordination
medium-center-api	58	35	Center operations
small-center-api	49	28	Center operations

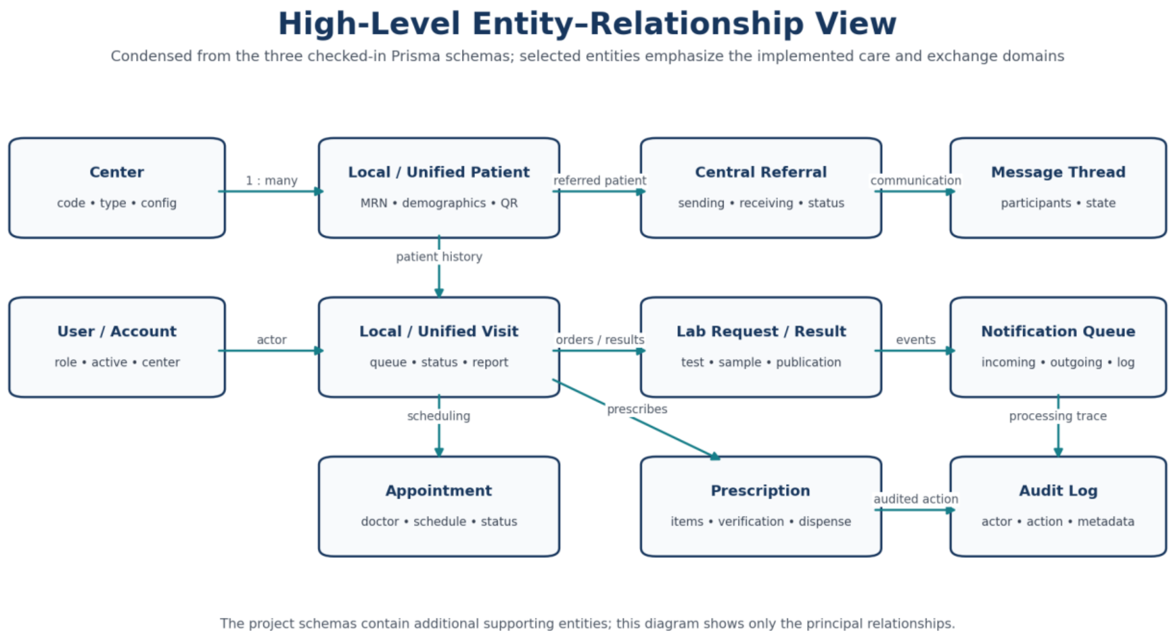

Figure 4.2. High-level entity–relationship view condensed from the checked-in Prisma schemas.

4.4 Authentication, Authorization, and Audit

Successful login produces a bearer JWT with a twelve-hour expiry. Authentication middleware verifies the token and loads the account identity. Authorization middleware checks permitted role values, while route queries constrain center-specific records using the authenticated workspace. Patient portal routes derive patient identity from the authenticated account rather than trusting arbitrary client identifiers. Passwords and reset codes are hashed with bcrypt. Reset codes use six digits, a ten-minute validity window, a maximum of five attempts, and a short-lived reset token. Zod validates request shapes; Helmet sets defensive HTTP headers; CORS reads configured origins; and sensitive paths record actor, action, entity, metadata, and request context in audit logs.

Security boundary: UI route hiding is a usability measure, not the security boundary. The server-side JWT, role checks, patient/center scoping, validation, and audit code provide the enforceable controls. A production release still needs automated negative-authorization tests and operational security controls.

4.5 Inter-Center Referral Design

A sending center can request a referral. Selection logic considers receiving capabilities recorded centrally, including doctor, operating-room, medicine, and load/availability information. The workflow then supports receiving-manager review, doctor assignment, visit creation, completion, cancellation, or return with a reason. Explicit enum values make the cross-center state auditable. Legacy status values remain declared for backward compatibility.

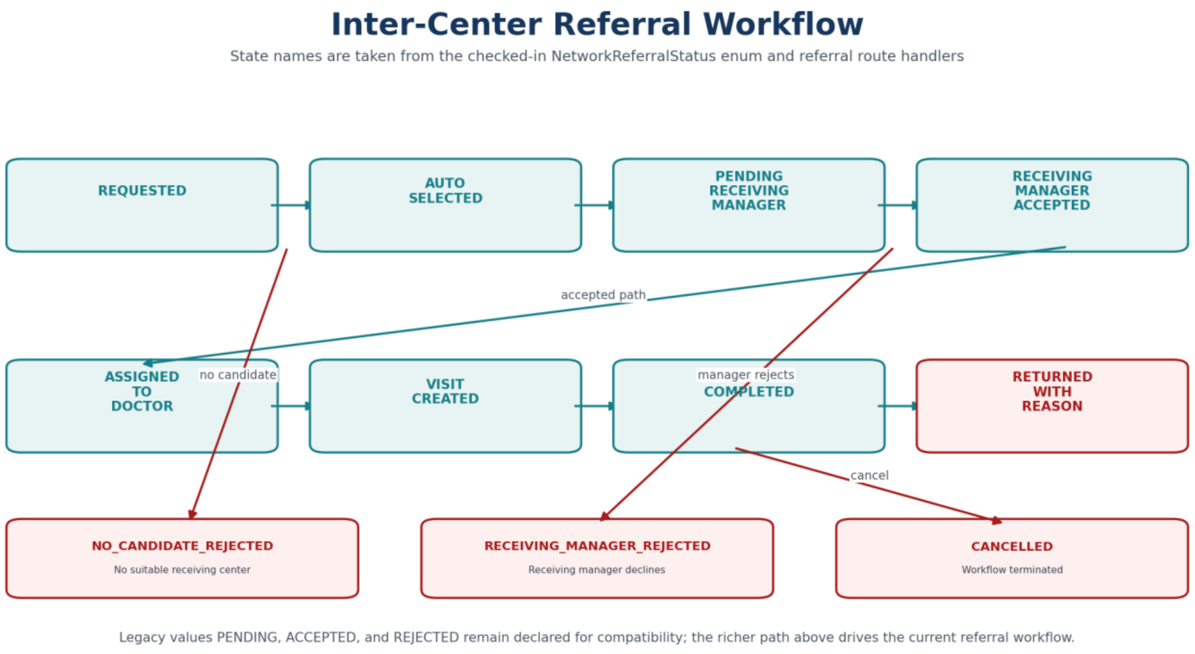

Figure 4.3. Inter-center referral workflow.

4.6 Visit and Queue Design

The medium-center workflow models reception, triage, doctor waiting, active treatment, laboratory waiting, pharmacy waiting, report upload, central upload, completion, and cancellation. It can associate nursing assessments, workflow tasks, laboratory requests, prescriptions, and result reports. The small-center workflow removes triage, laboratory, and pharmacy stages from its configured role experience and follows a shorter doctor-treatment-upload path. This distinction reflects center capability rather than two differently styled copies of the same screen.

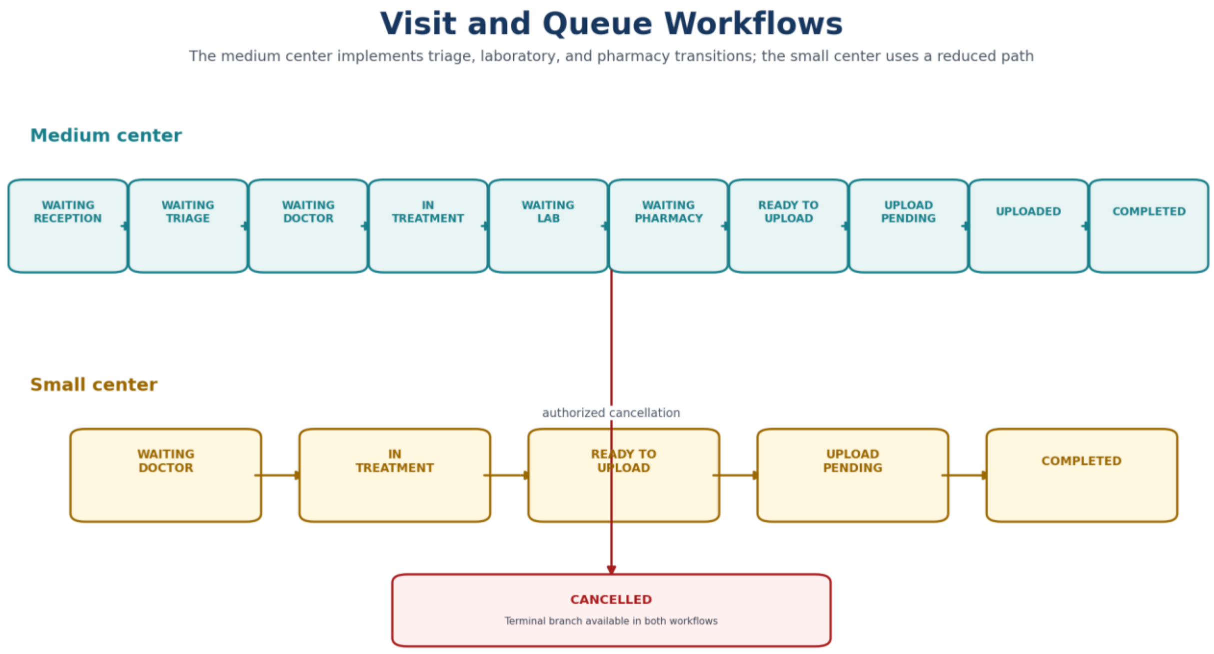

Figure 4.4. Medium- and small-center visit/queue state models.

4.7 Notification and Synchronization Design

Each API starts a background processor that checks notification queues at an approximately sixty-second interval. Local APIs process outgoing notifications for their configured center; the central API processes network records and delivery back to centers. Status, attempts, next-attempt timing, error fields, communication logs, and processing logs support inspection and retry. The mechanism is asynchronous polling and database-backed exchange, not a real-time event broker.

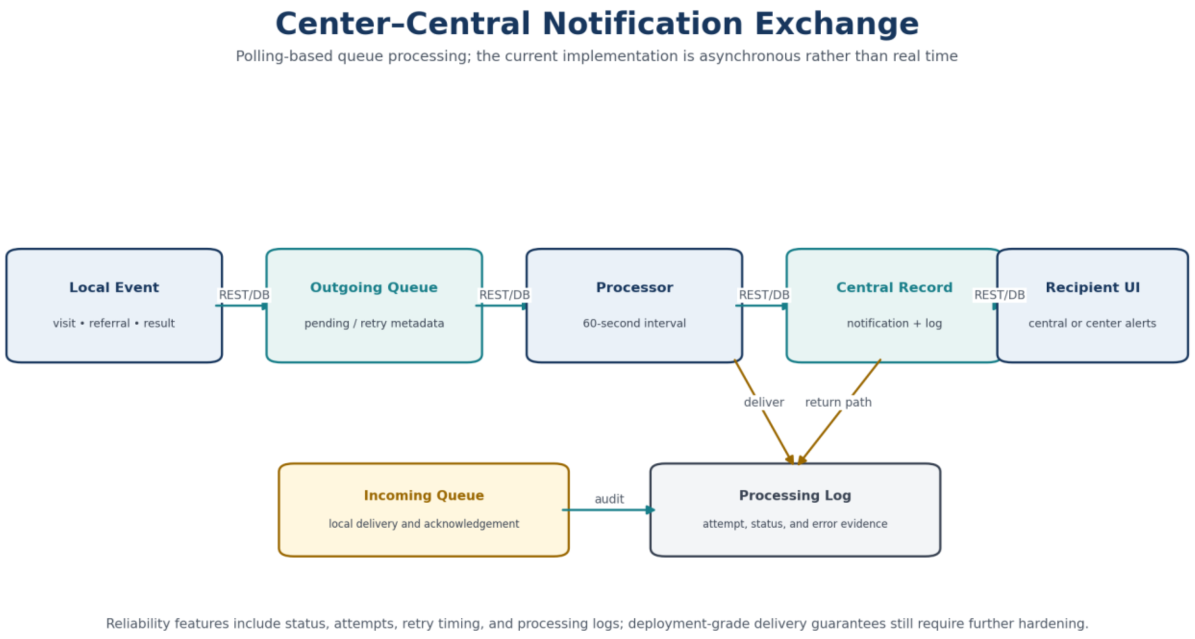

Figure 4.5. Notification and synchronization flow.

4.8 Appointment Suggestions, QR Access, and Care Assistant

Appointment suggestions use 30-minute slots between 09:00 and 17:00, search up to fourteen days, return up to five suggestions, and reject conflicts with active appointments. Patient QR cards use UUID-like tokens and a healthcare-patient payload format; authenticated center routes resolve the token and audit access. The rendered QR image currently depends on an external QR-image service, which introduces availability and privacy considerations. The care assistant applies a medical-topic guard and returns structured urgency, summary, actions, follow-up questions, red flags, self-care, and disclaimer fields using configured OpenRouter or Gemini providers or a local fallback. It is explicitly a prototype and not a diagnostic model.

4.9 Design Limitations

- Central source contains visit, laboratory, and pharmacy page components, but current central allowed-route configuration does not expose them to the administrator.
- Small-center source contains some generic laboratory/pharmacy components and models, but the current small-center role configuration exposes manager, doctor, receptionist, and patient workflows only.
- The three APIs repeat substantial schema and route code. This supports local specialization but creates synchronization and migration-consistency risk.
- Current exchange is polling-based and lacks a broker-backed delivery contract, idempotency specification, or formal service-level objective.
- Current JSON APIs are project-specific and do not claim HL7 FHIR conformance.

Chapter 5: Implementation

5.1 Implementation Overview

The implementation combines role-specific web and mobile interfaces with three Express APIs and PostgreSQL databases accessed through Prisma. The figures in this chapter were selected to show the overall design, the main professional roles, and the differences between central, medium-center, small-center, and mobile workflows. Repetitive filters, forms, and minor interface states were omitted.

5.2 Central Administration System

The central portal gives the network administrator access to center management, shared master data, analytics, reports, notifications, and audit records. It provides a network-level view without replacing the operational systems used inside each center.

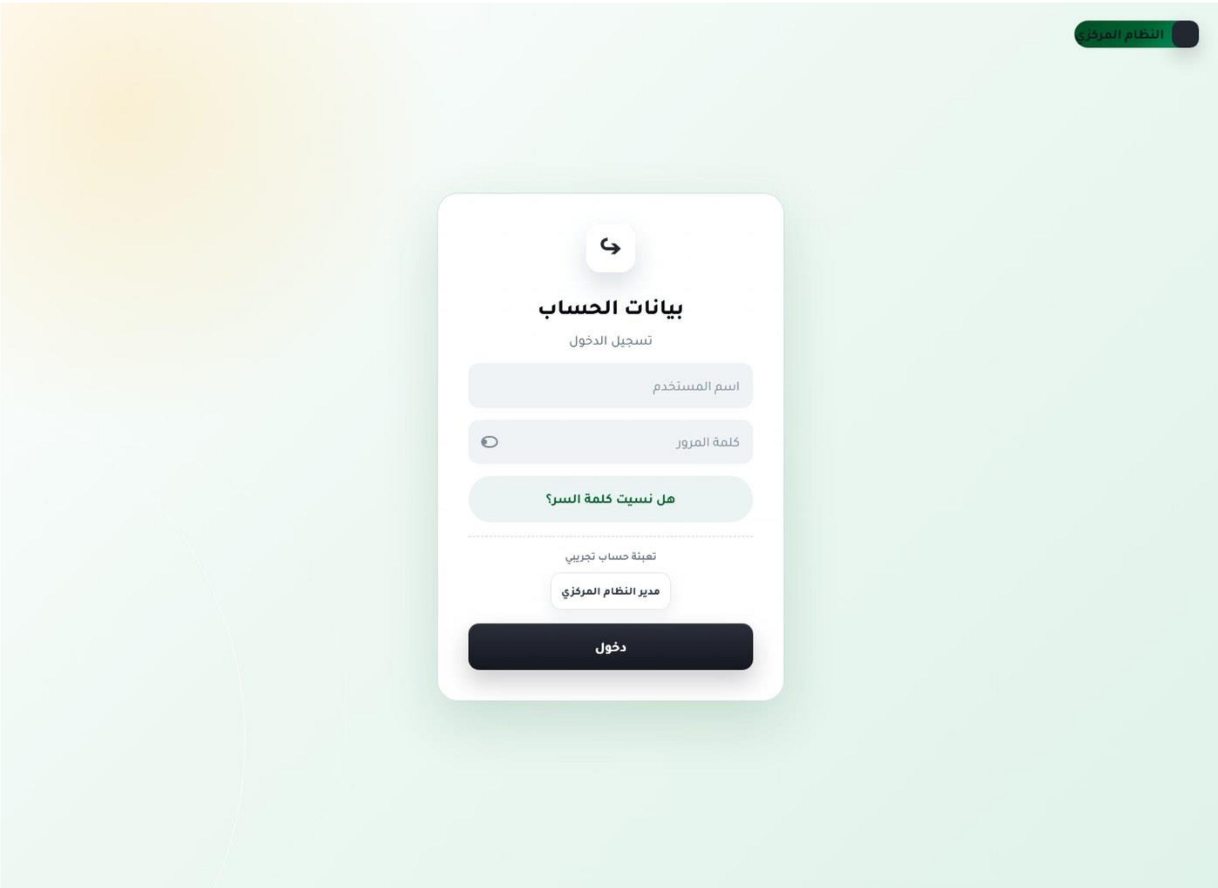

Figure 5.1. Sign-in and account recovery — Unauthenticated user.

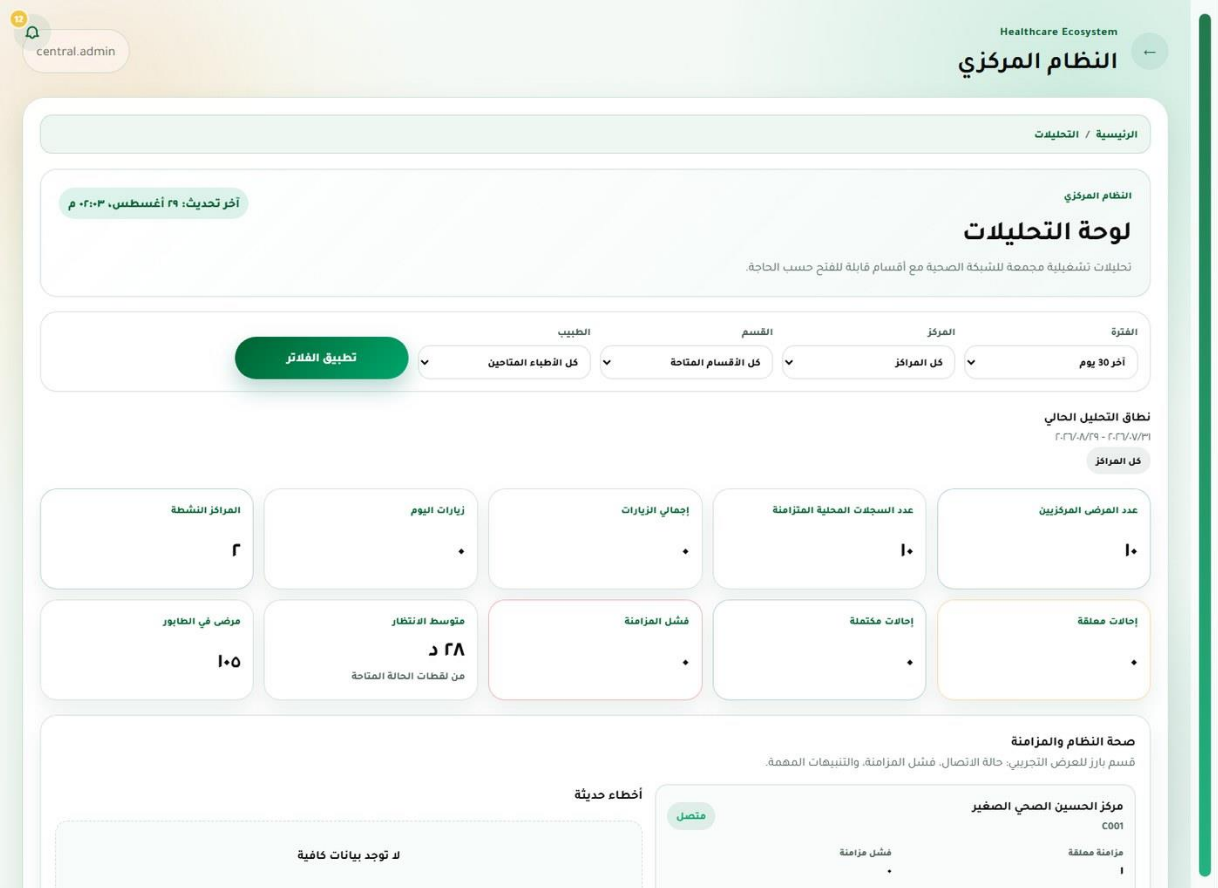

Figure 5.2. Analytics dashboard — Central administrator.

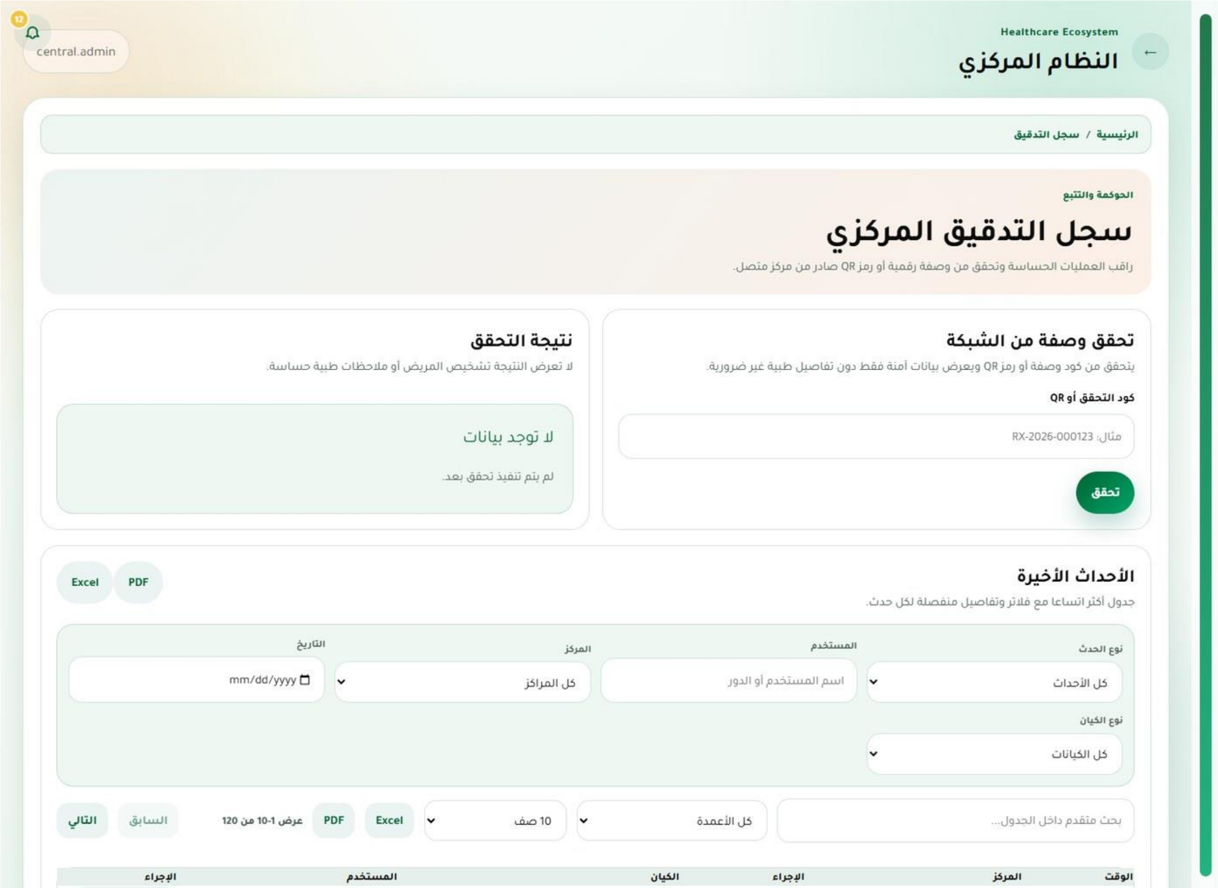

Figure 5.3. Audit logs and prescription verification — Central administrator.

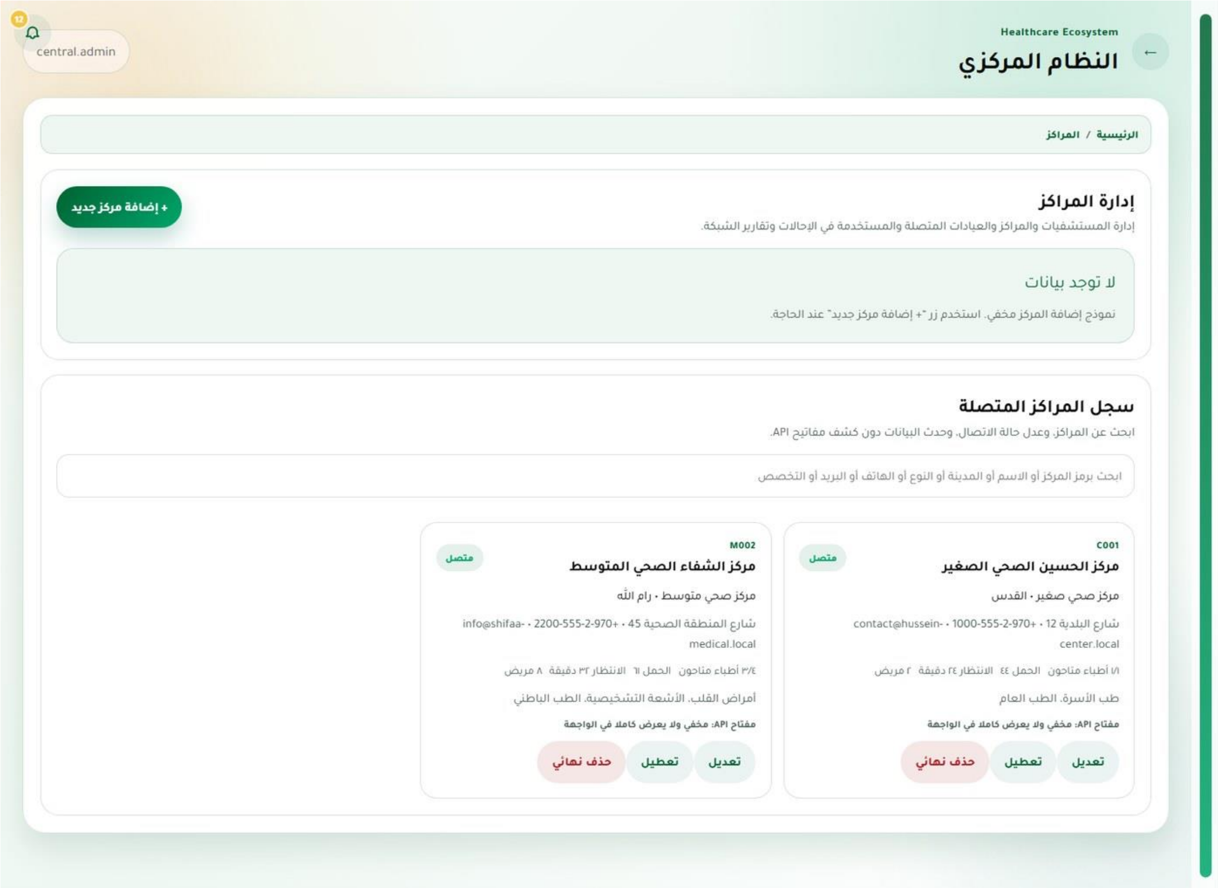

Figure 5.4. Healthcare-center administration — Central administrator.

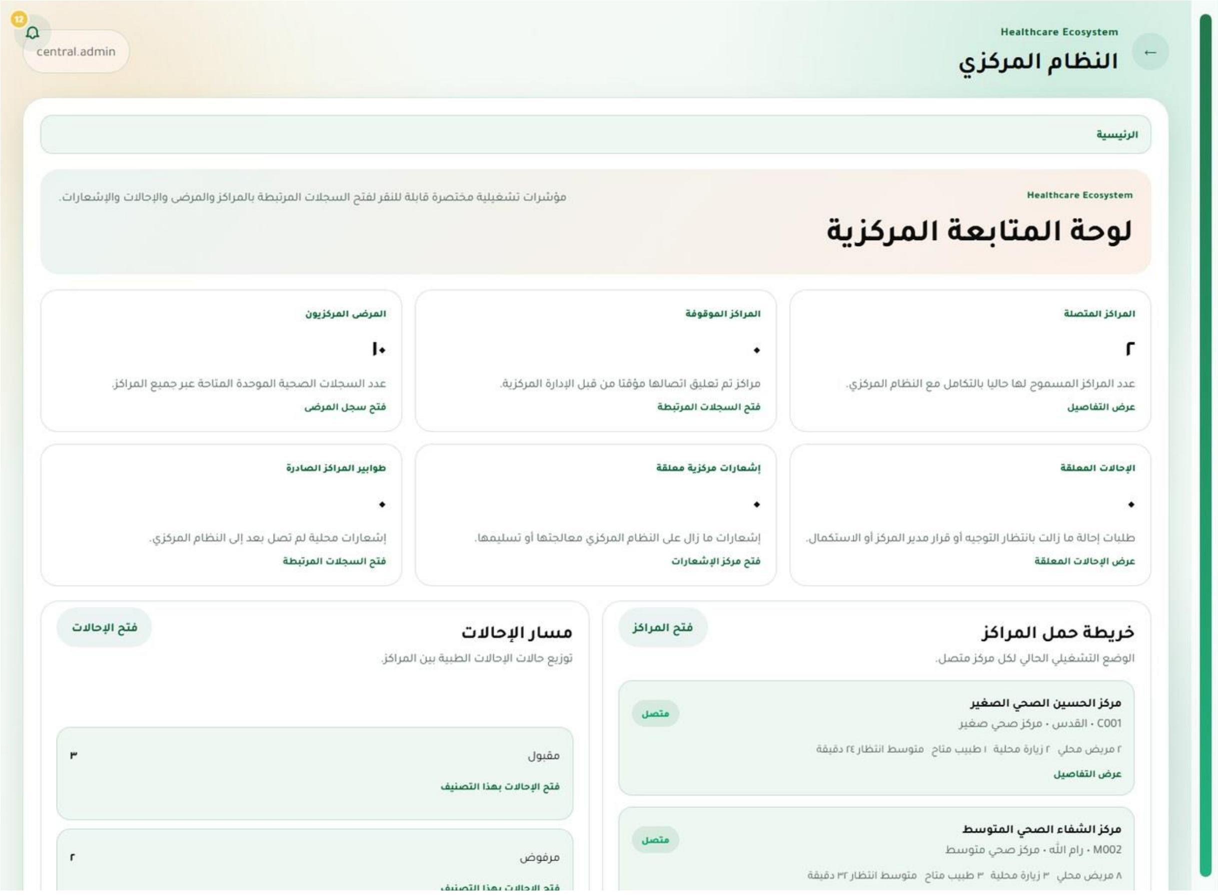

Figure 5.5. Role-aware dashboard — Central administrator.

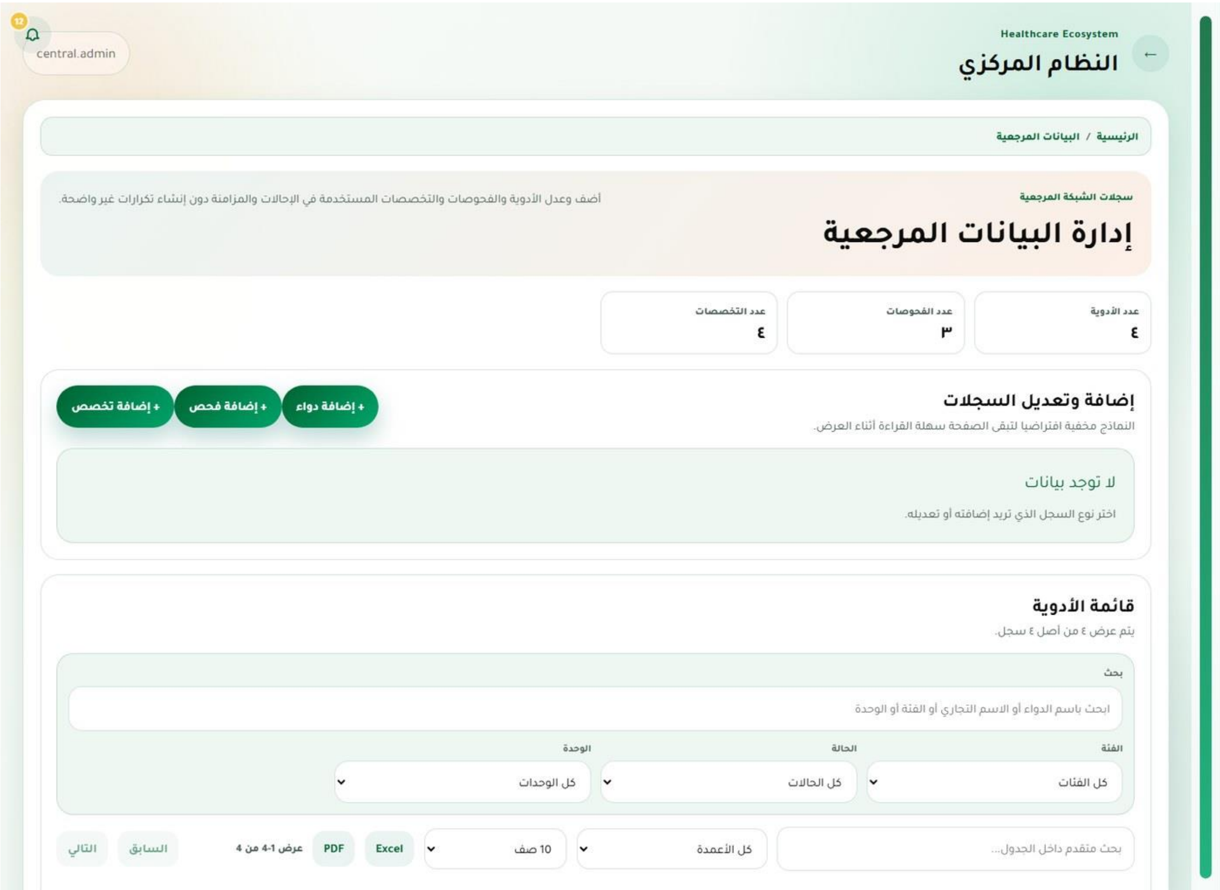

Figure 5.6. Central master data — Central administrator.

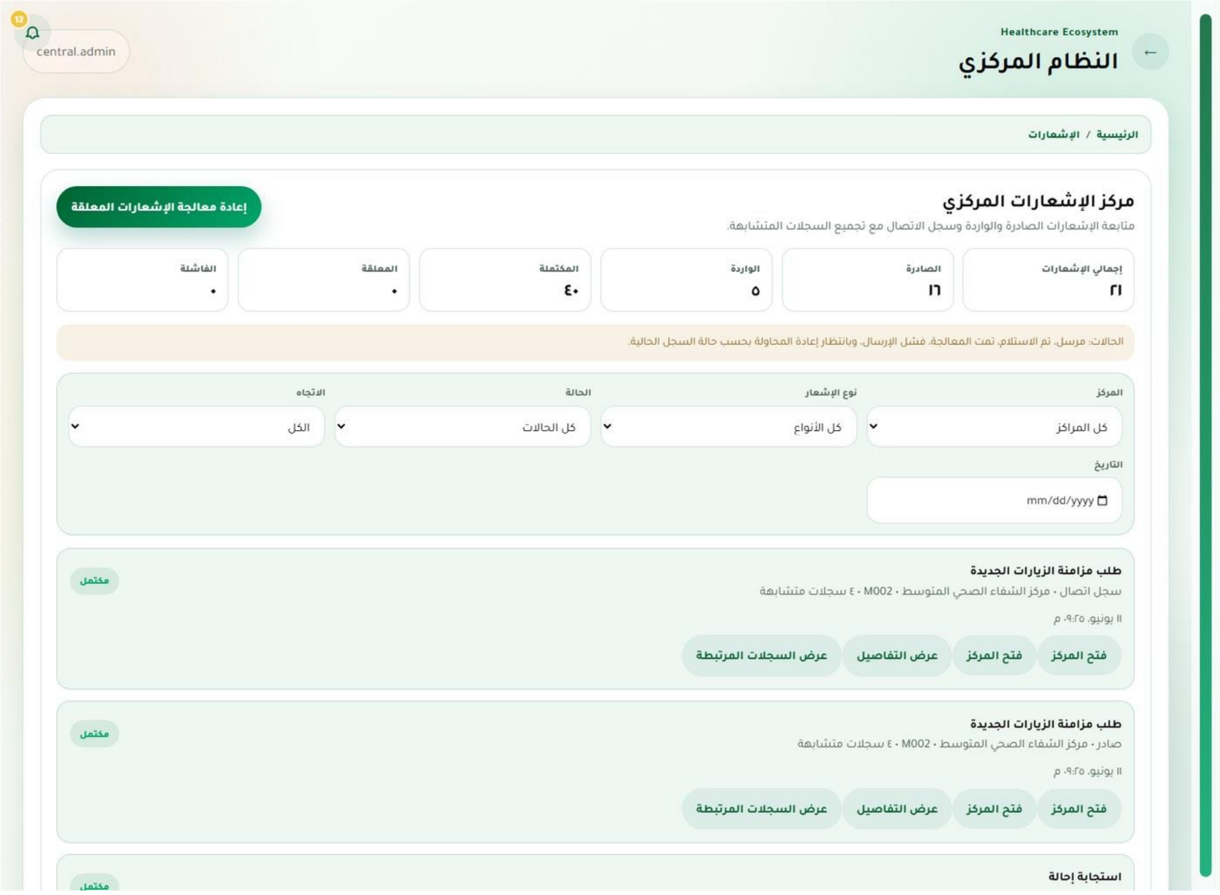

Figure 5.7. Notifications and synchronization — Central administrator.

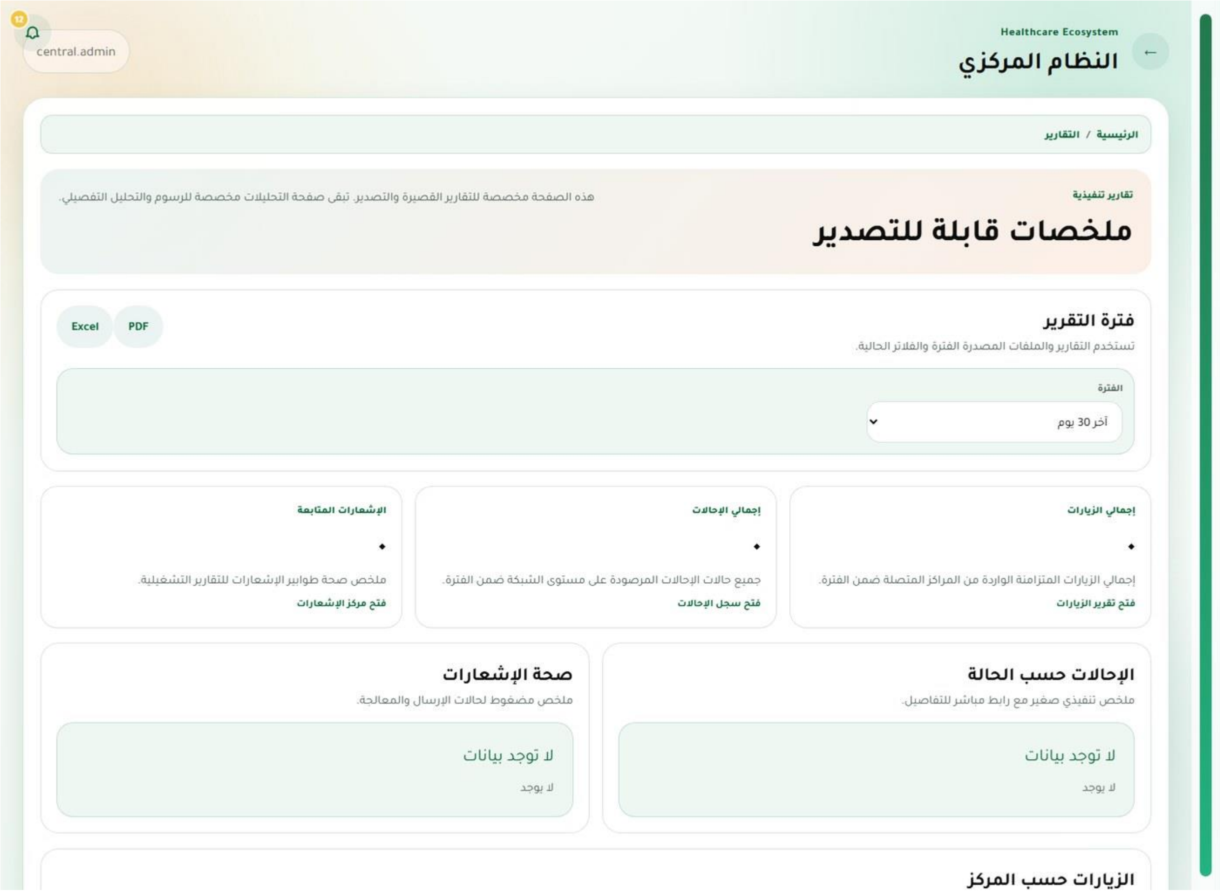

Figure 5.8. Operational reports — Central administrator.

5.3 Medium-Center Web System

The medium-center web portal provides the broadest local workflow. Managers, doctors, receptionists, nurses, laboratory staff, pharmacists, and patients receive navigation and actions that match their responsibilities.

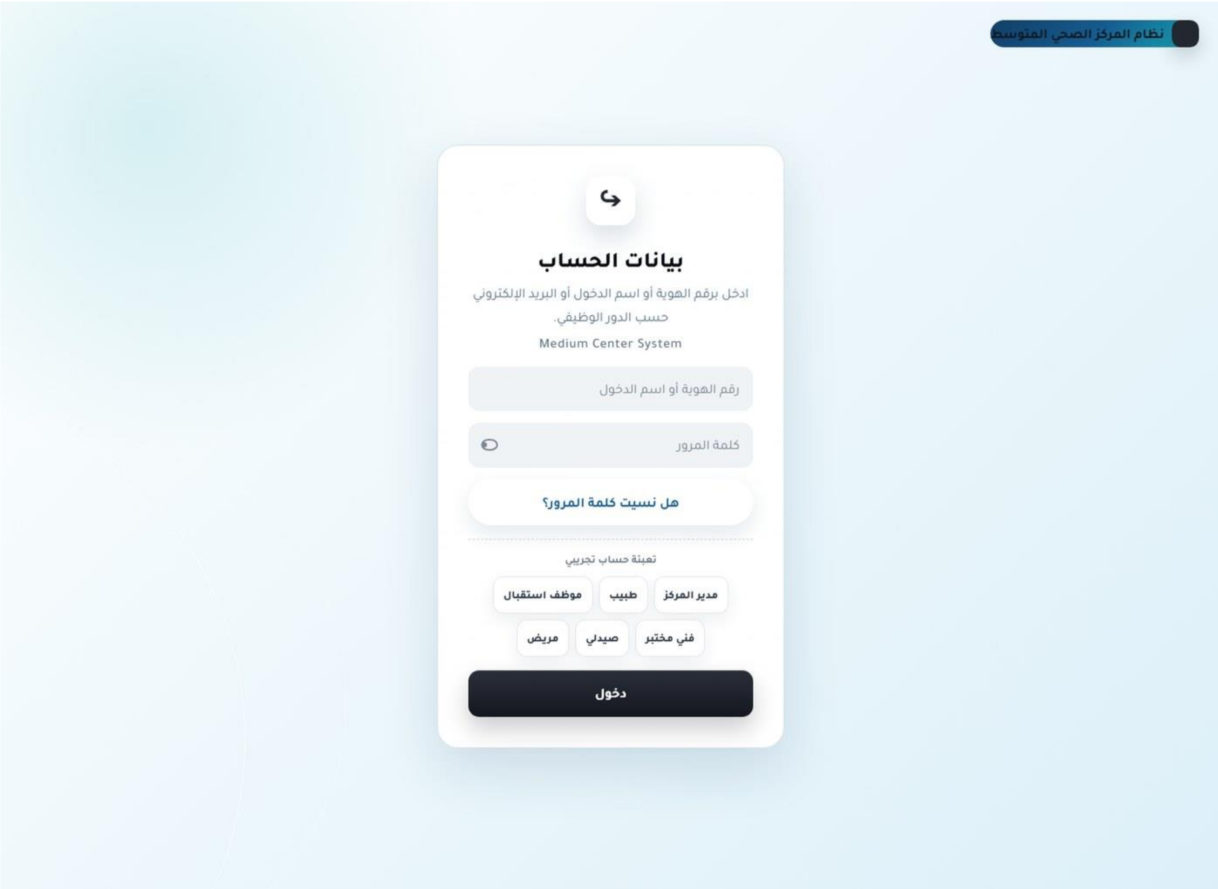

Figure 5.9. Sign-in and account recovery — Unauthenticated user.

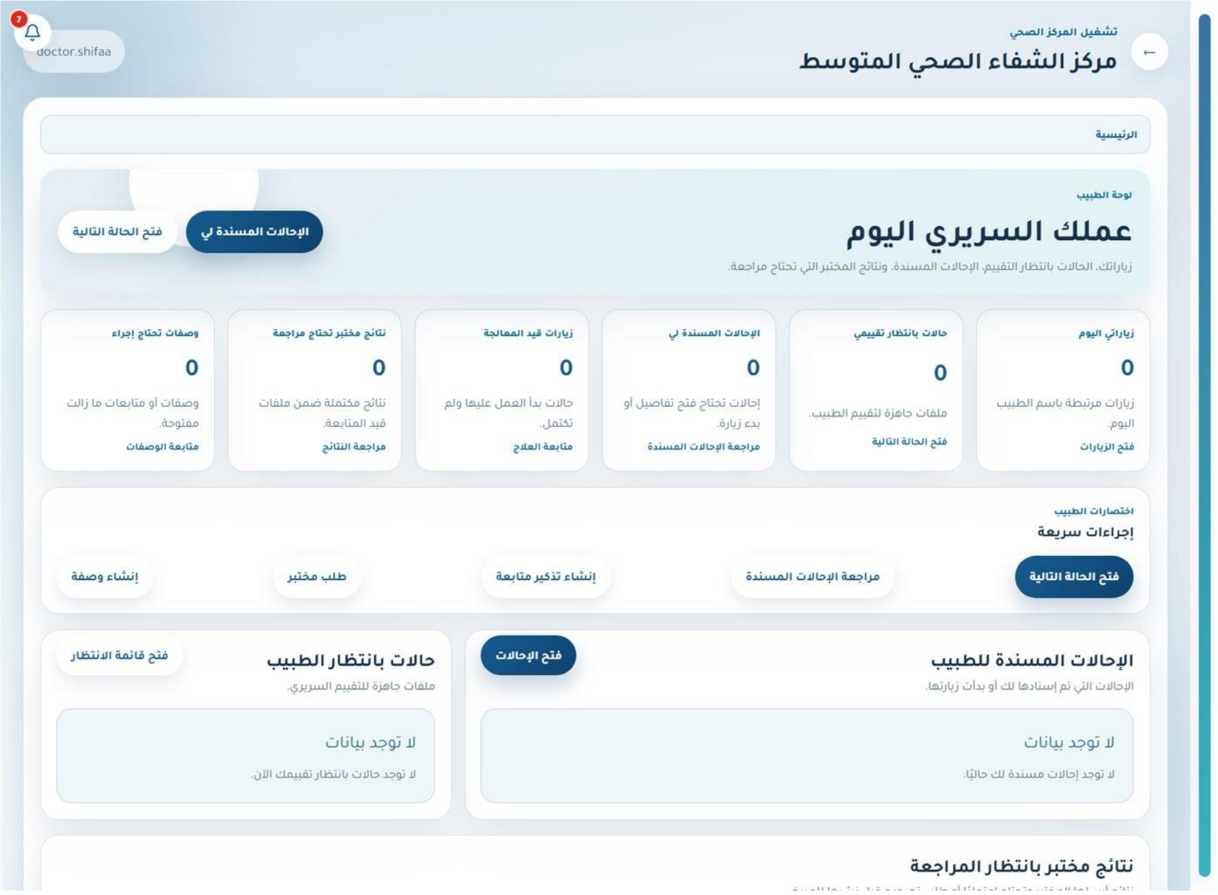

Figure 5.10. Role-aware dashboard — Doctor.

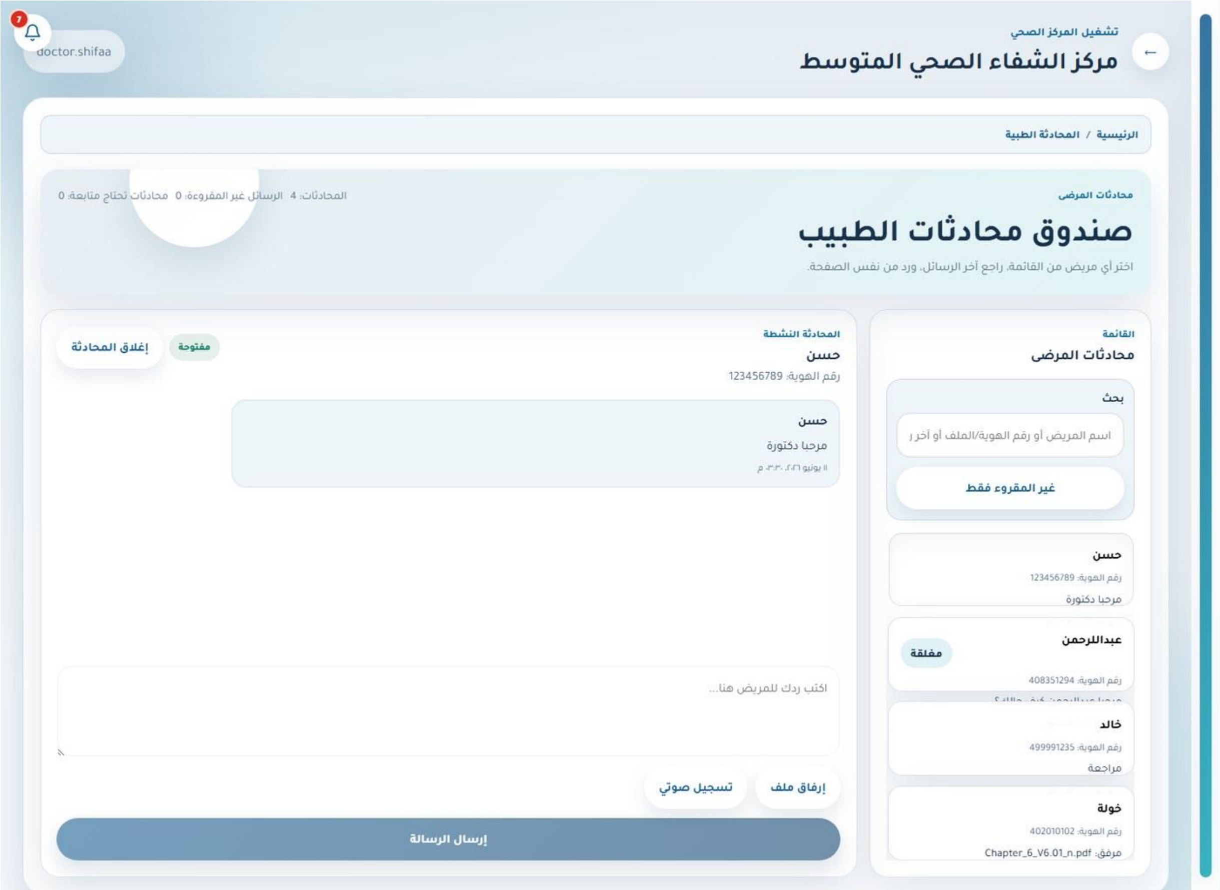

Figure 5.11. Secure patient–doctor messaging — Doctor.

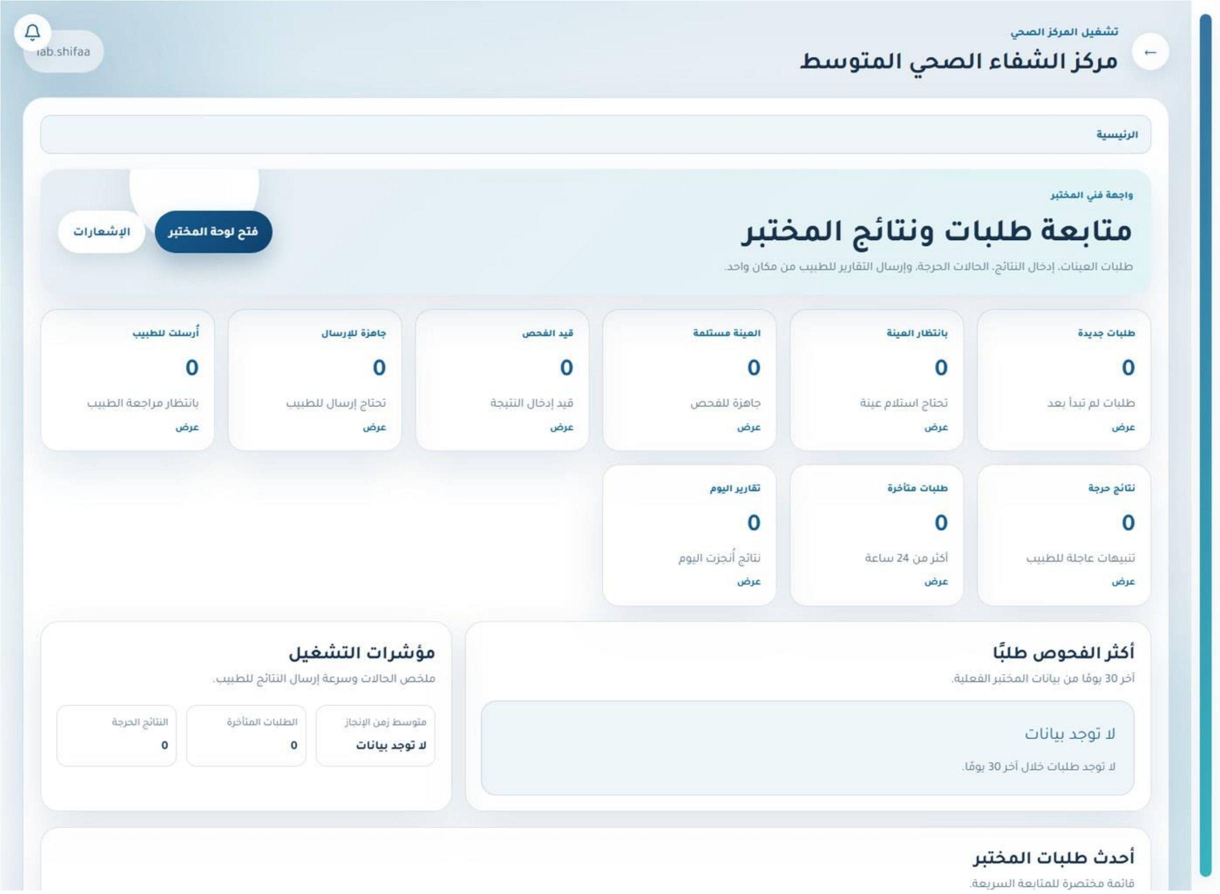

Figure 5.12. Role-aware dashboard — Laboratory staff.

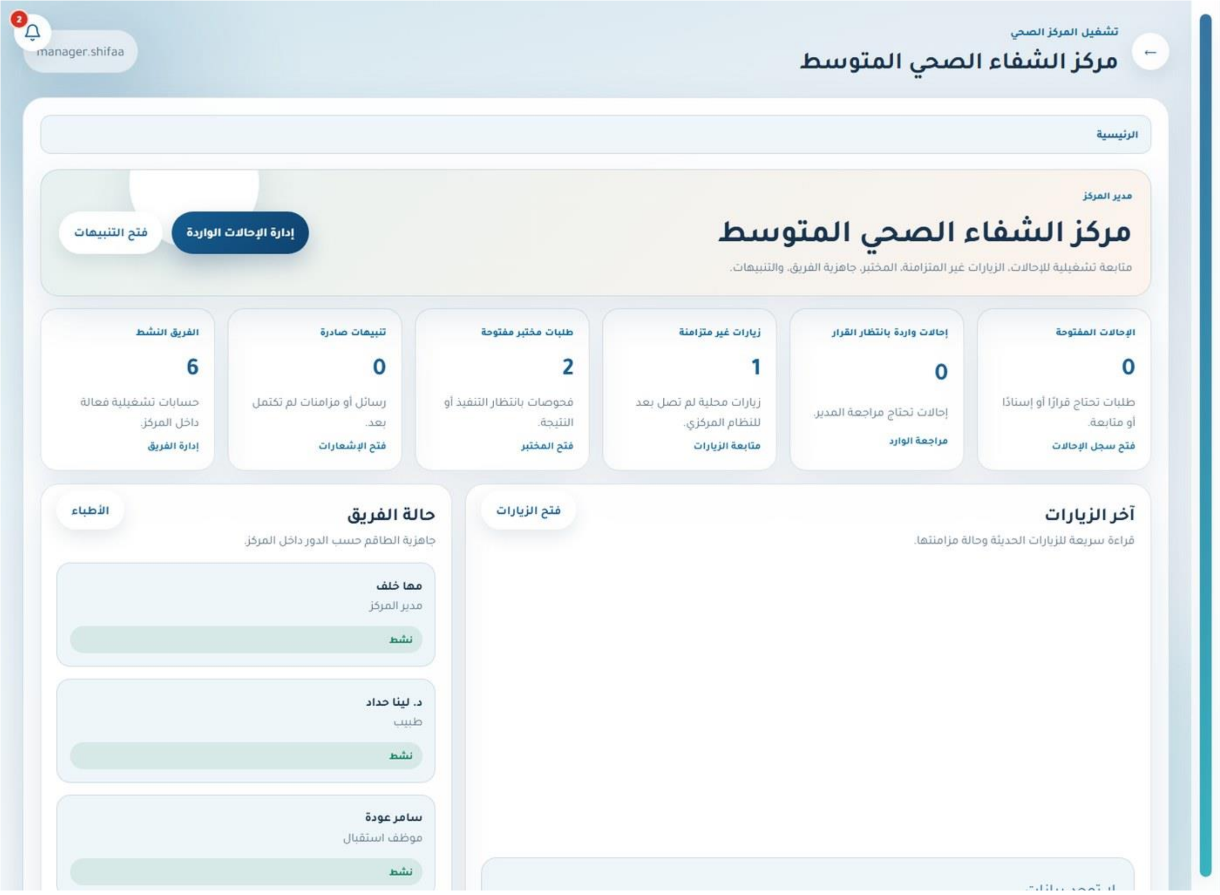

Figure 5.13. Role-aware dashboard — Center manager.

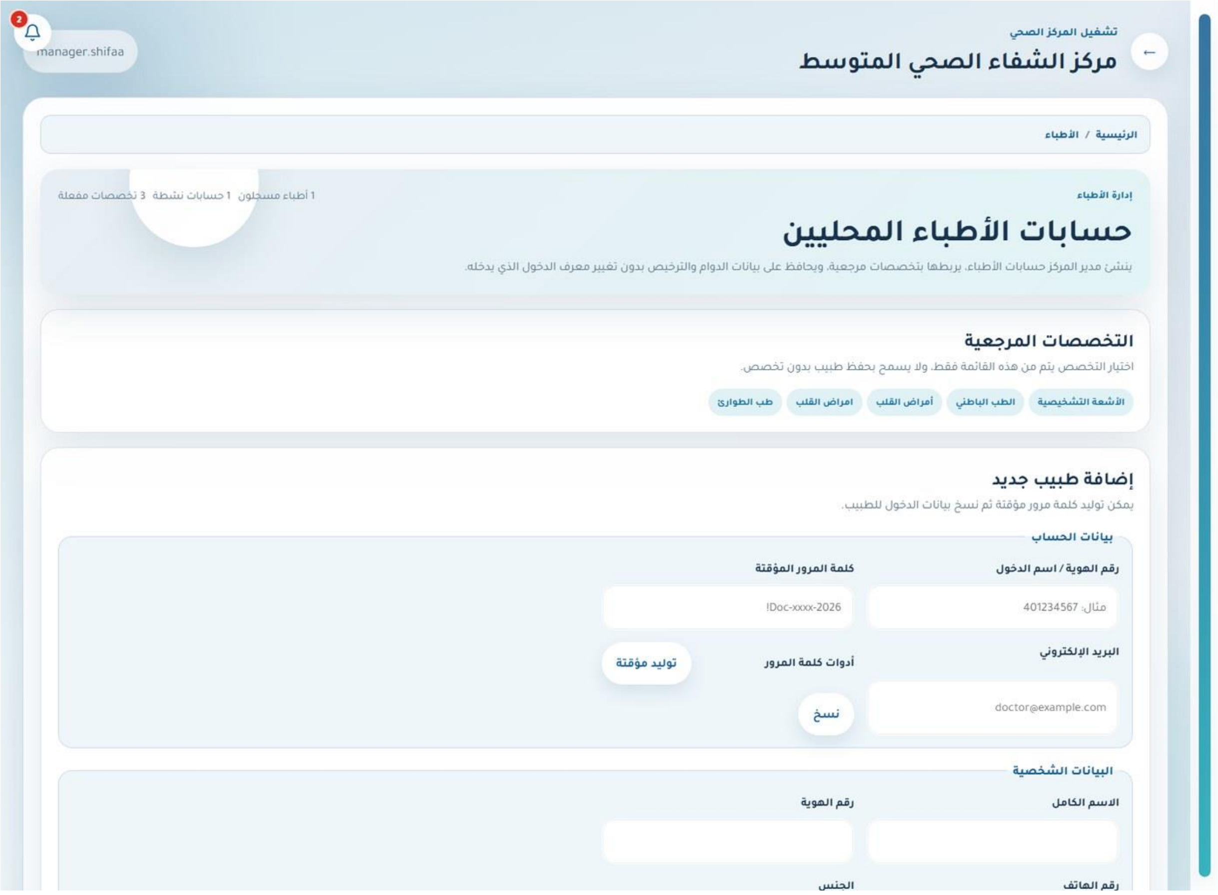

Figure 5.14. Doctor directory and availability — Center manager.

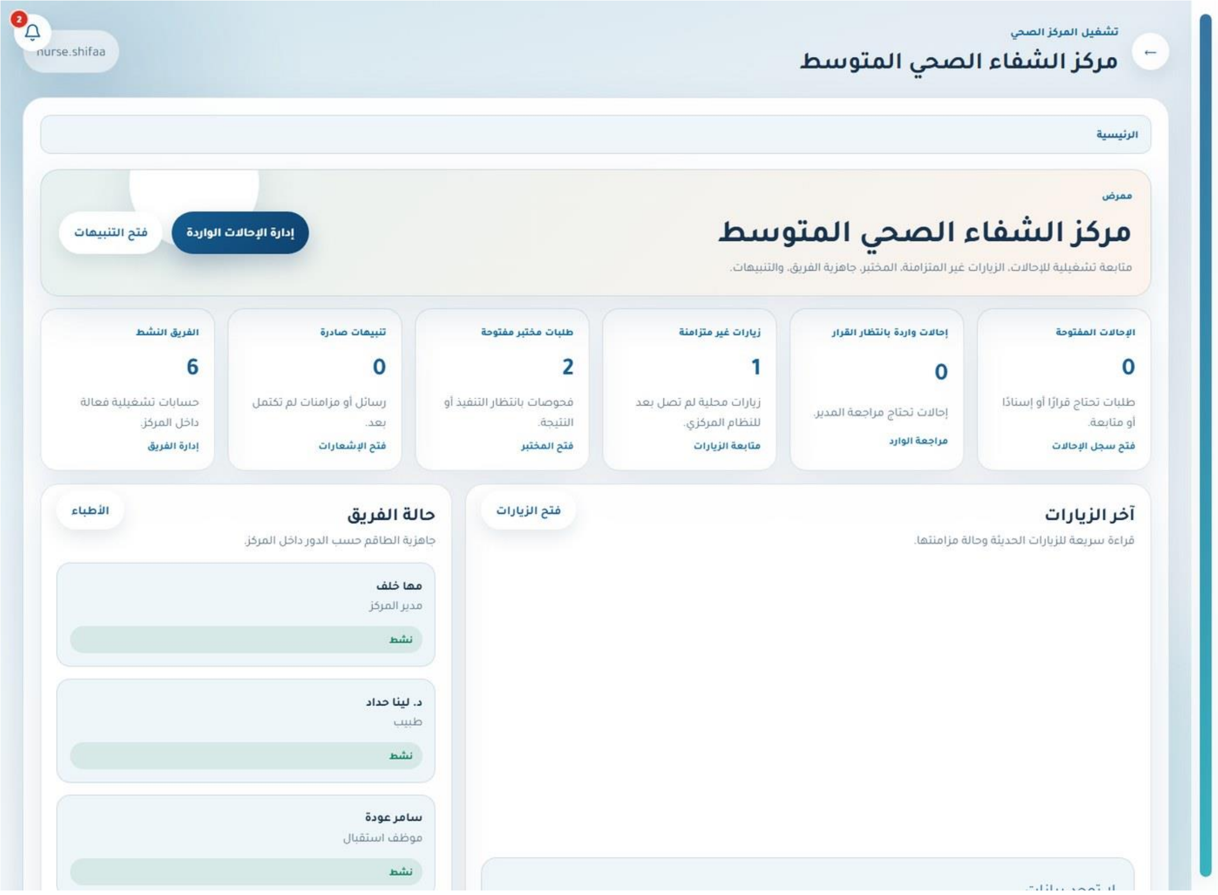

Figure 5.15. Role-aware dashboard — Nurse.

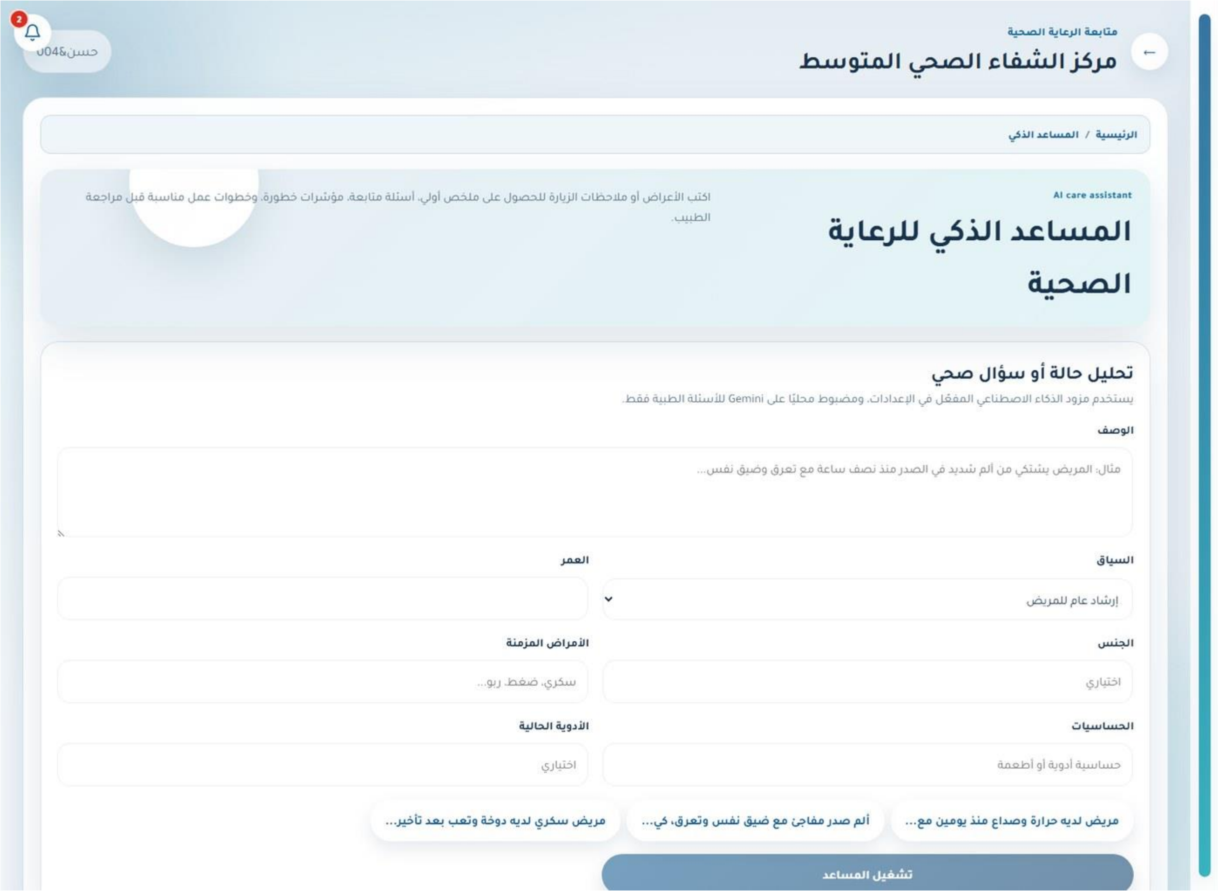

Figure 5.16. Prototype care assistant — Patient.

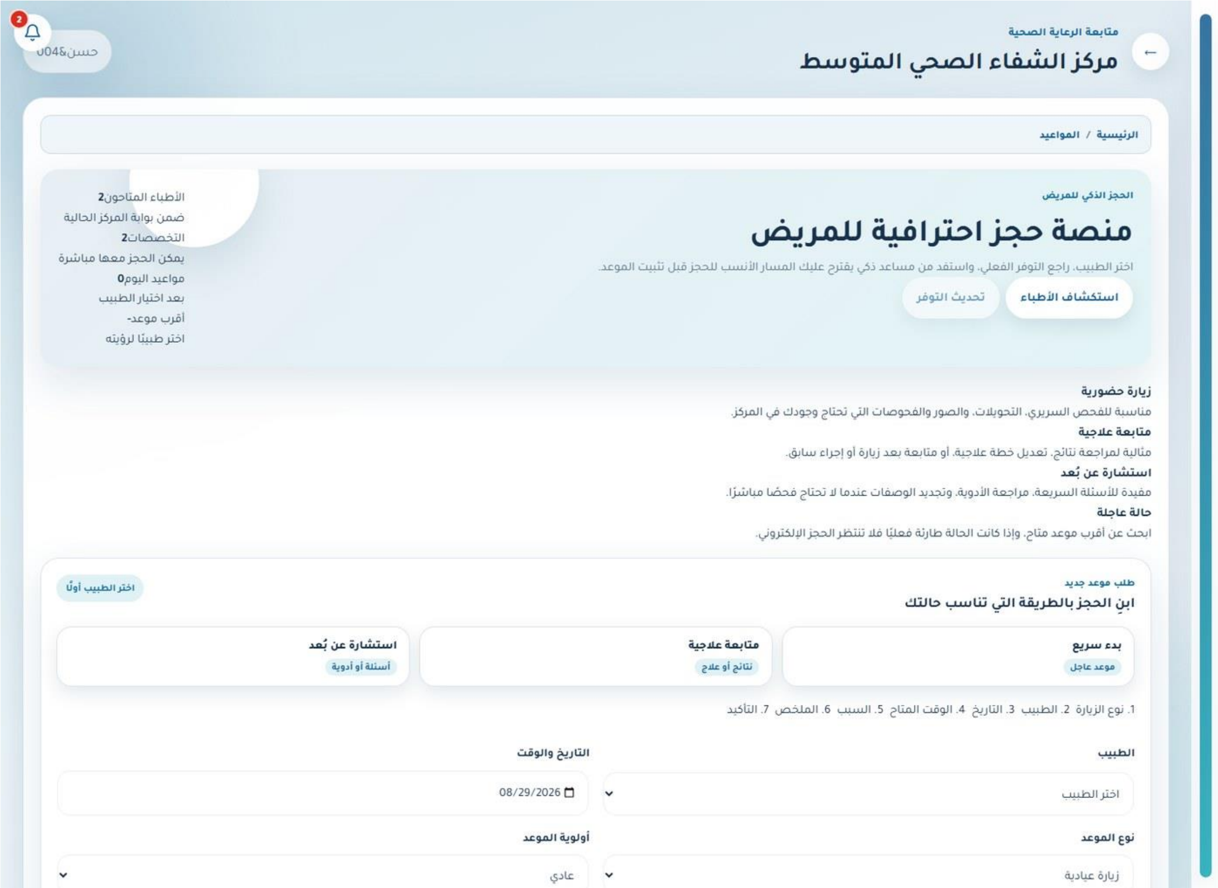

Figure 5.17. Appointments — Patient.

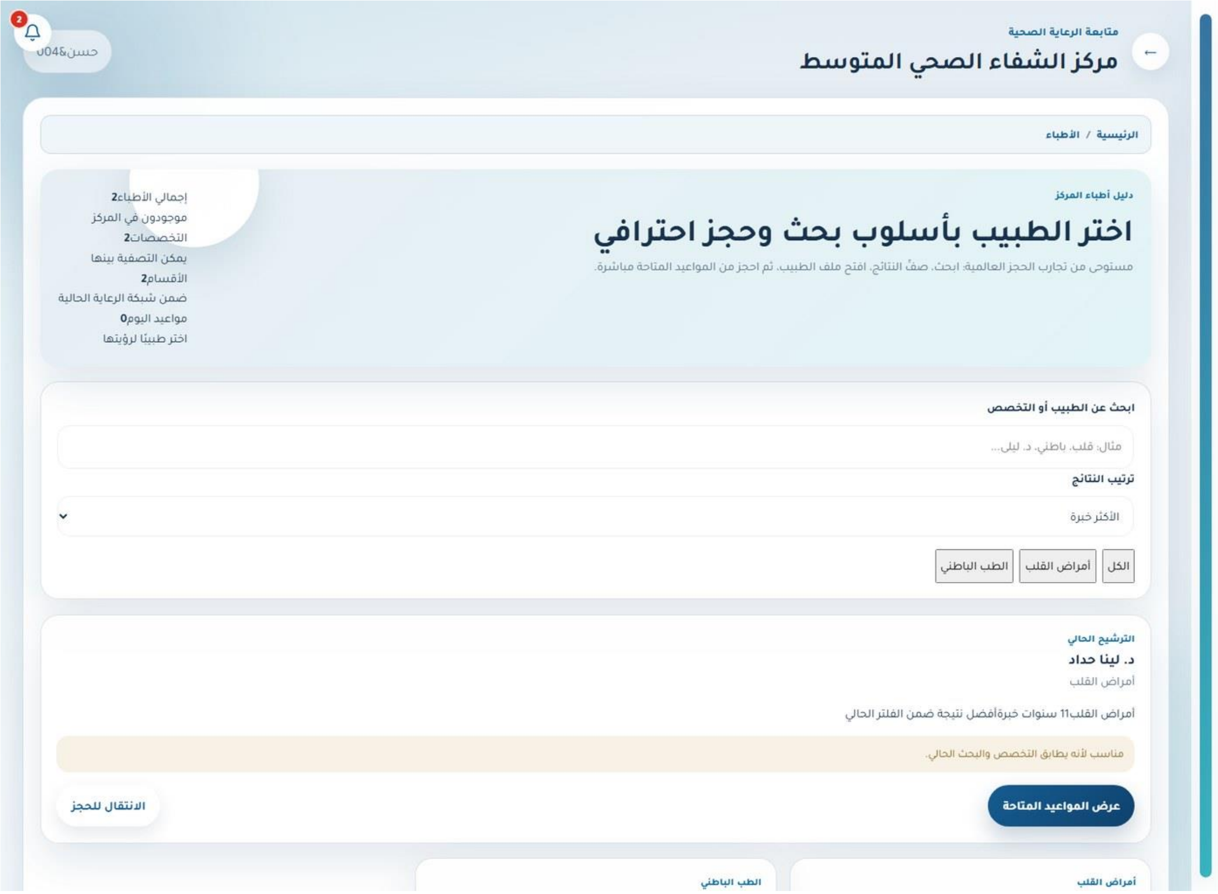

Figure 5.18. Doctor directory and availability — Patient.

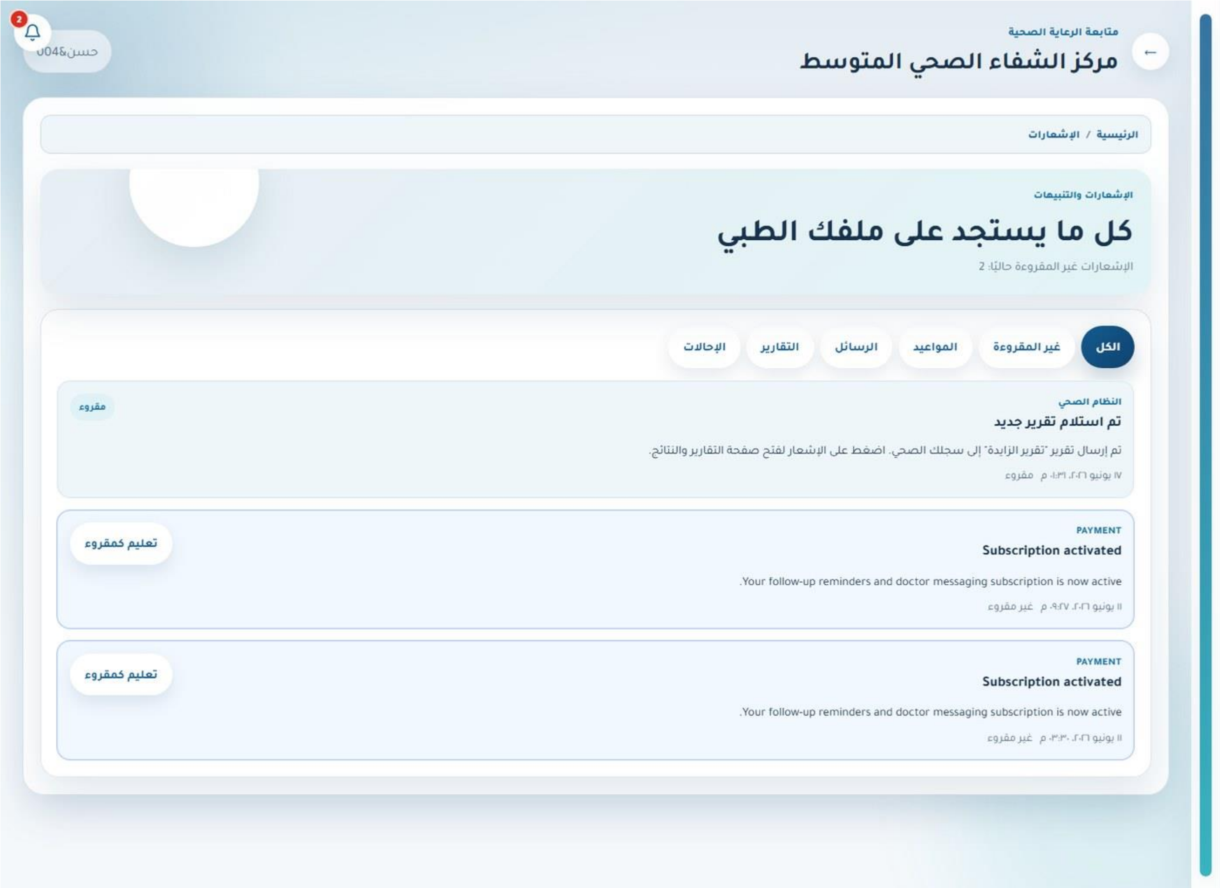

Figure 5.19. Notifications and synchronization — Patient.

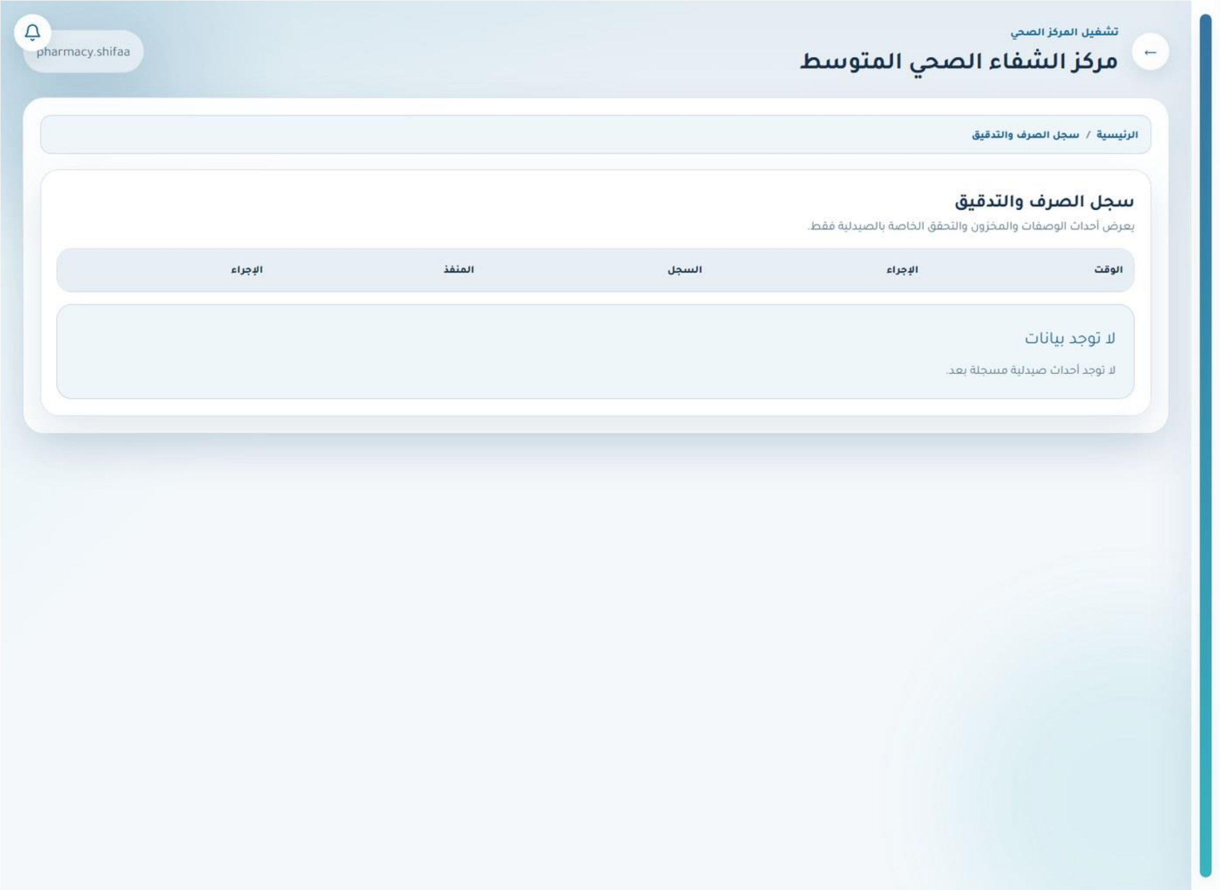

Figure 5.20. Pharmacy audit trail — Pharmacist.

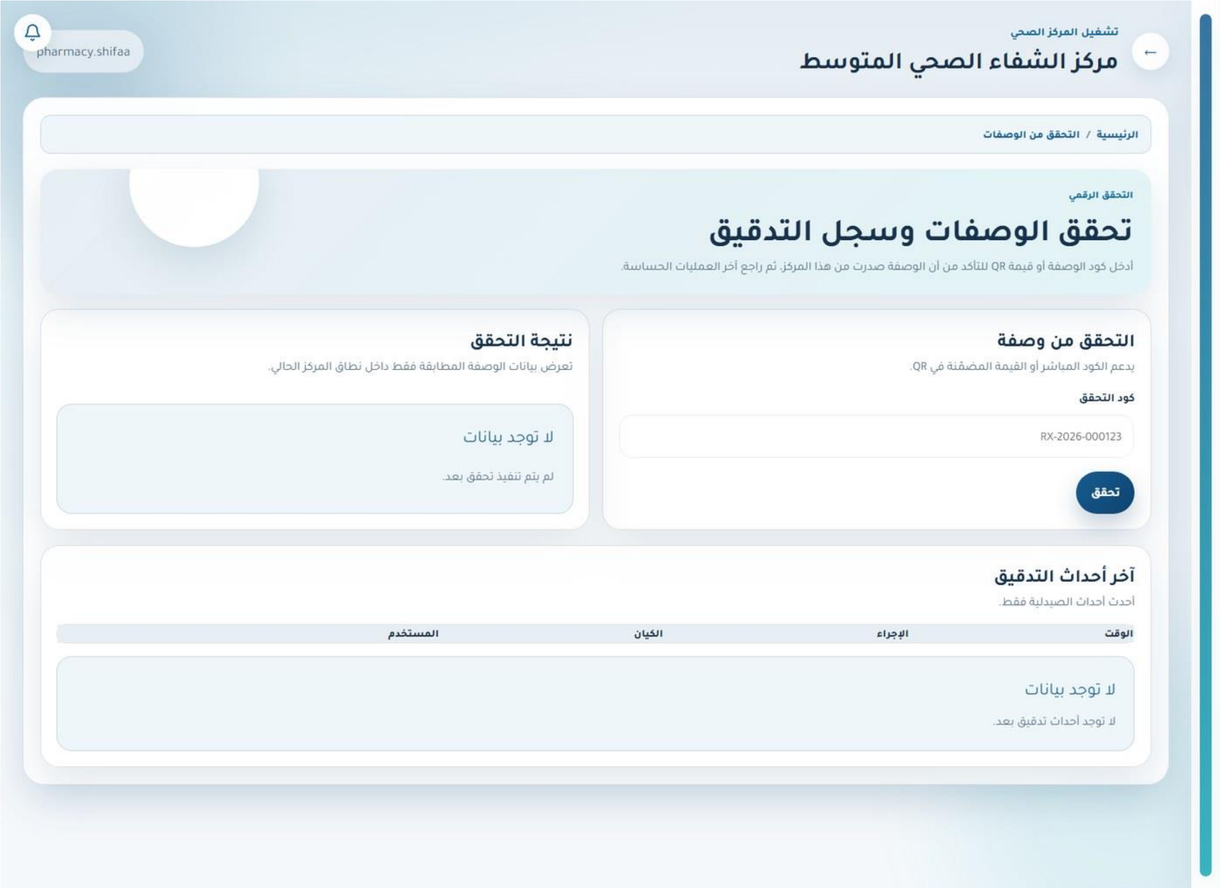

Figure 5.21. Prescription verification — Pharmacist.

Figure 5.22. Role-aware dashboard — Receptionist.

Figure 5.23. Patient QR card — Receptionist.

5.4 Small-Center Web System

The small-center web portal concentrates on manager, doctor, receptionist, and patient roles. It implements registration, doctor management, visit intake, a reduced treatment/upload workflow, referrals, prescription verification, appointments, medical record, messaging, notifications, and the prototype care assistant.

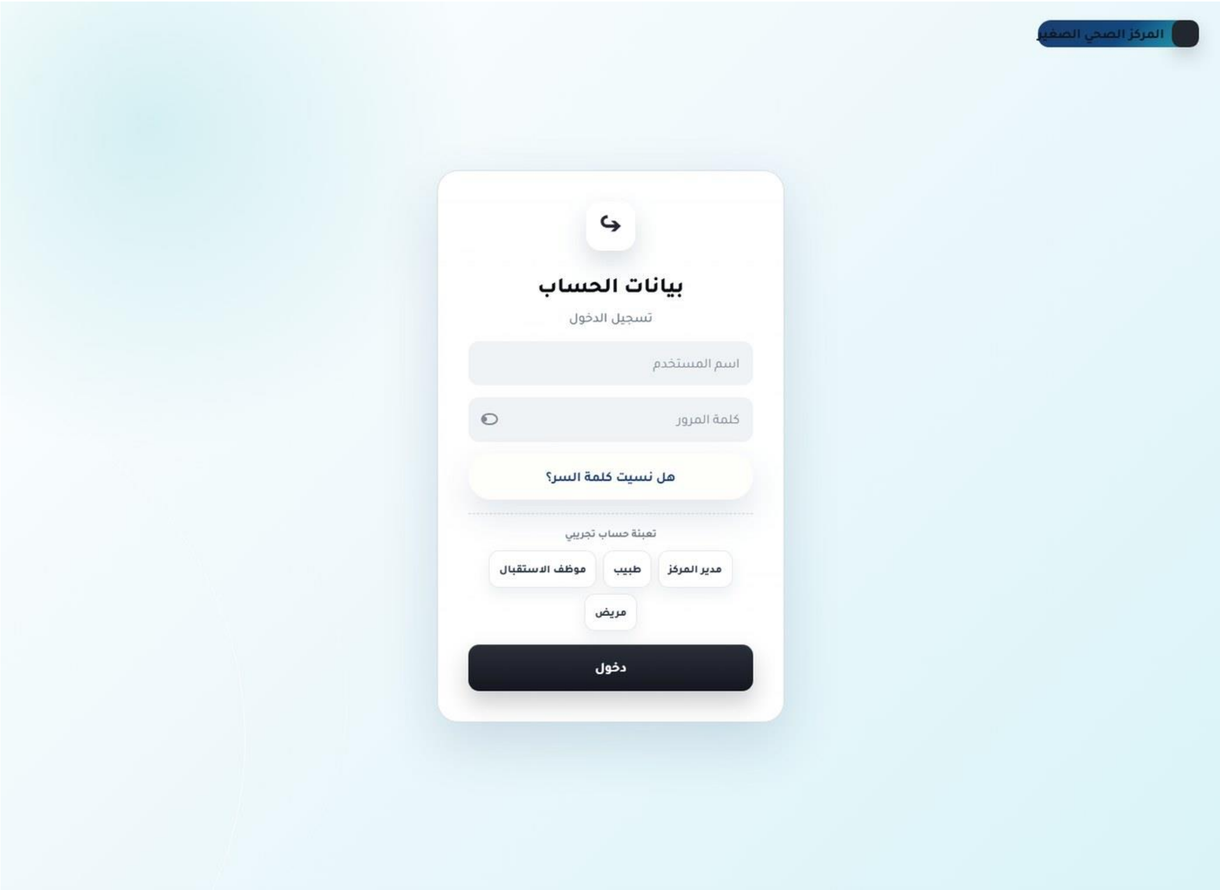

Figure 5.24. Sign-in and account recovery — Unauthenticated user.

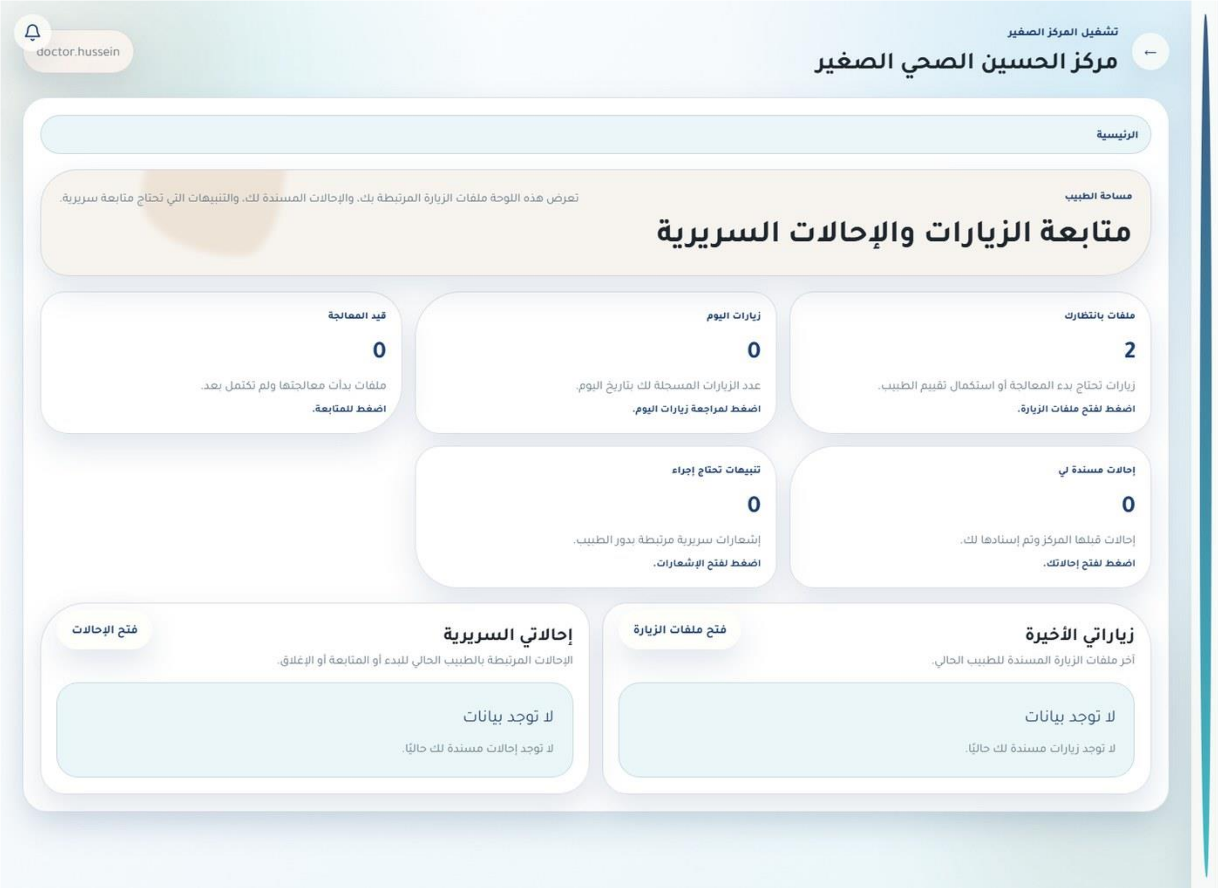

Figure 5.25. Role-aware dashboard — Doctor.

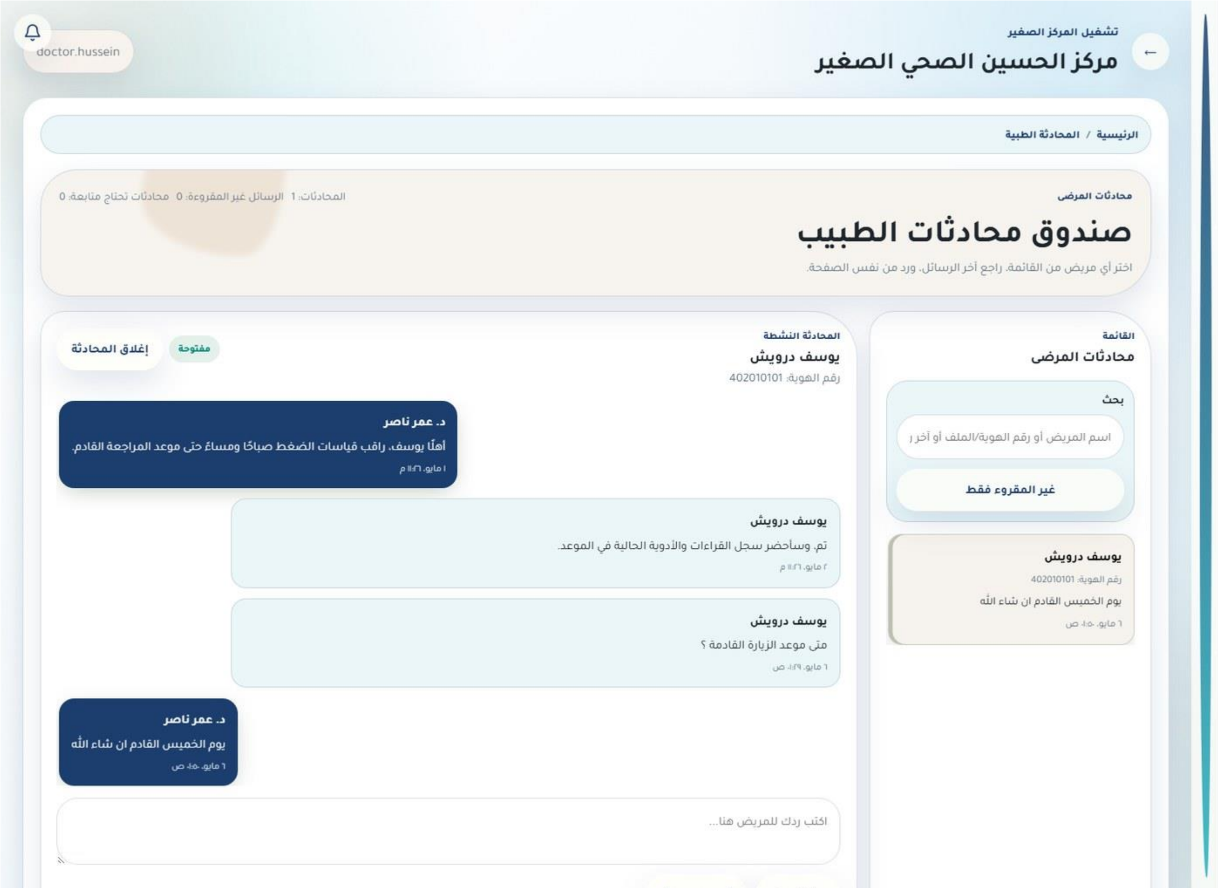

Figure 5.26. Secure patient–doctor messaging — Doctor.

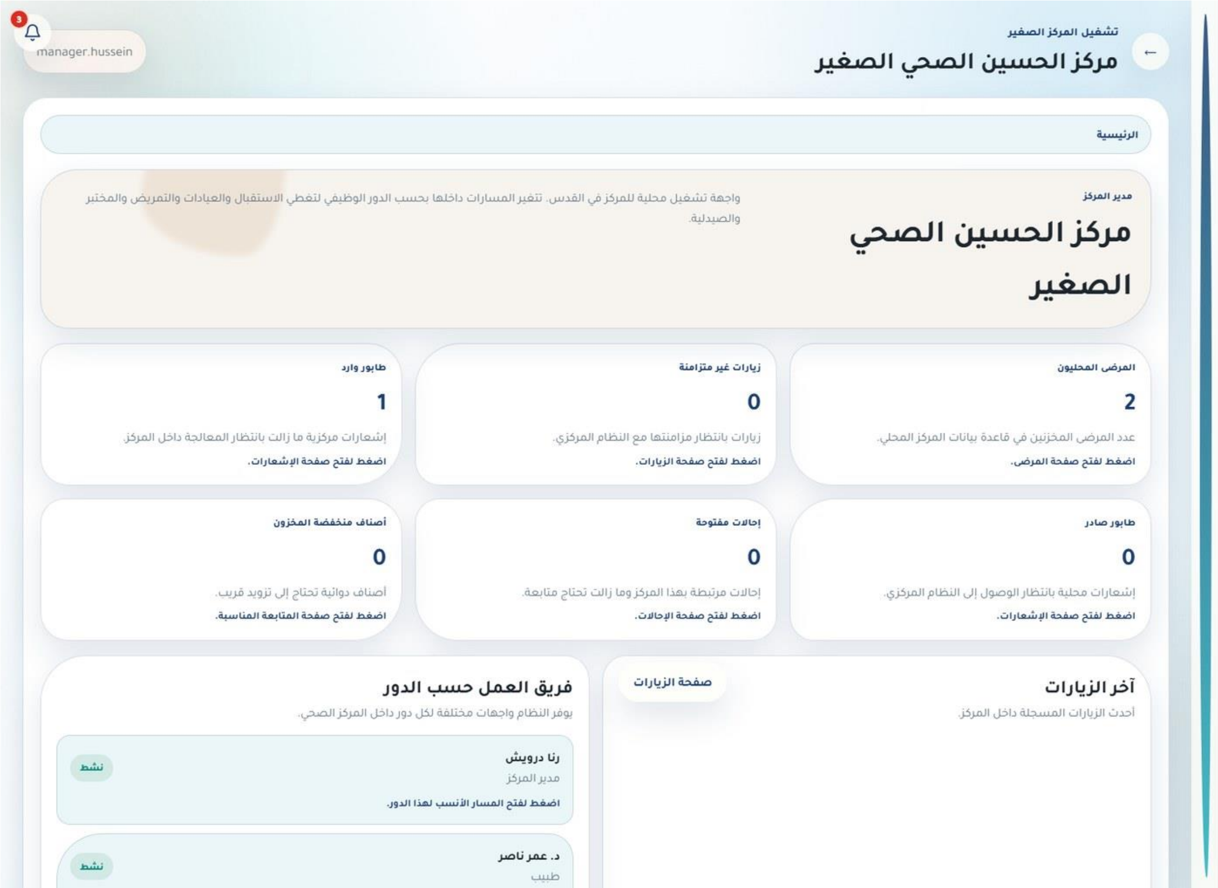

Figure 5.27. Role-aware dashboard — Center manager.

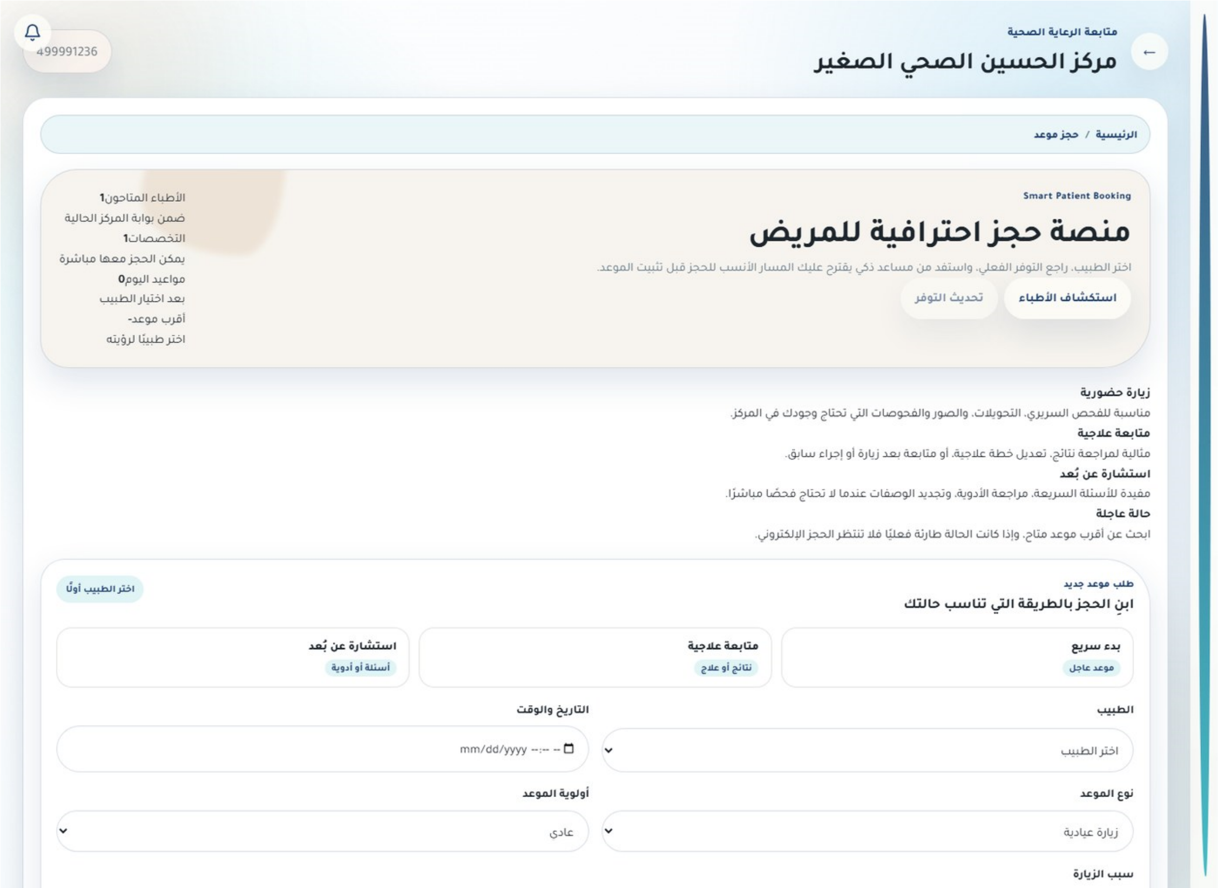

Figure 5.28. Appointments — Patient.

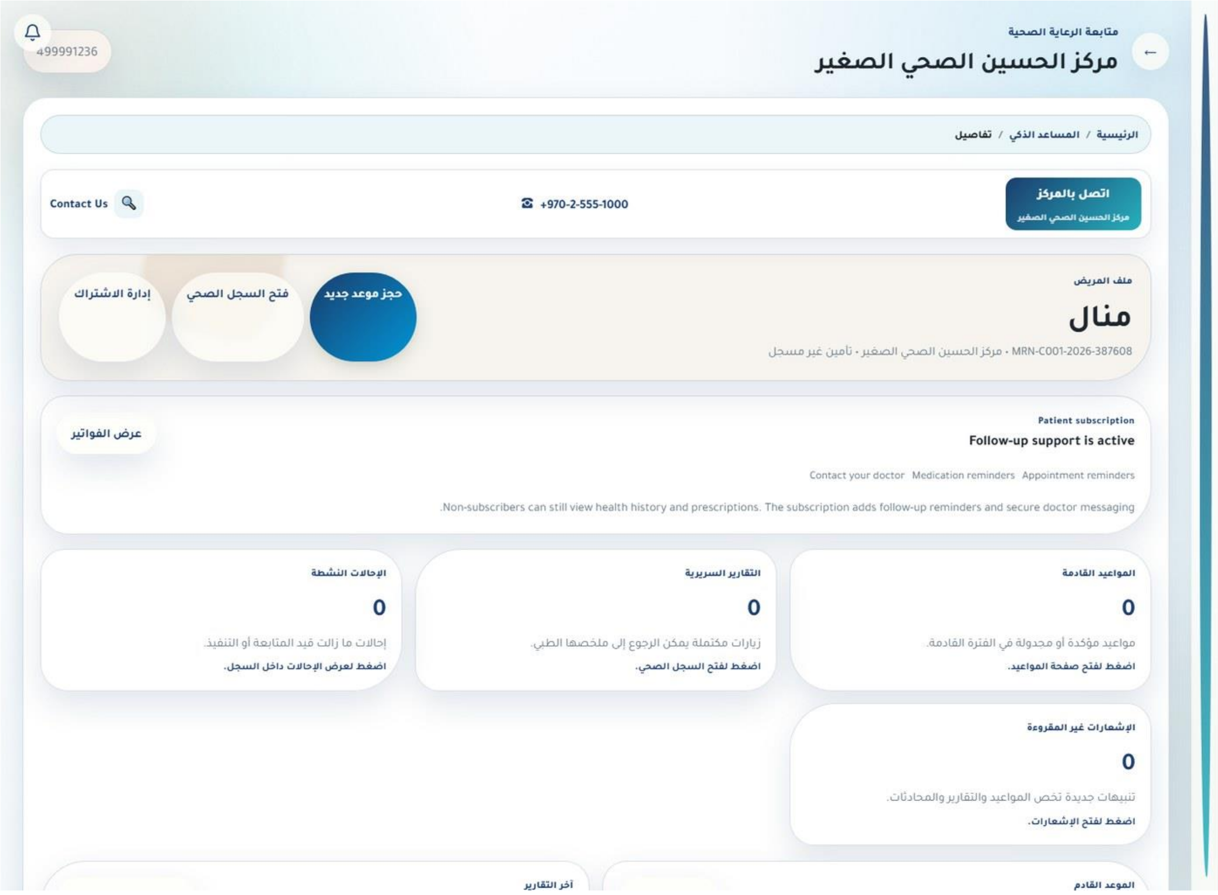

Figure 5.29. Mobile home dashboard — Patient.

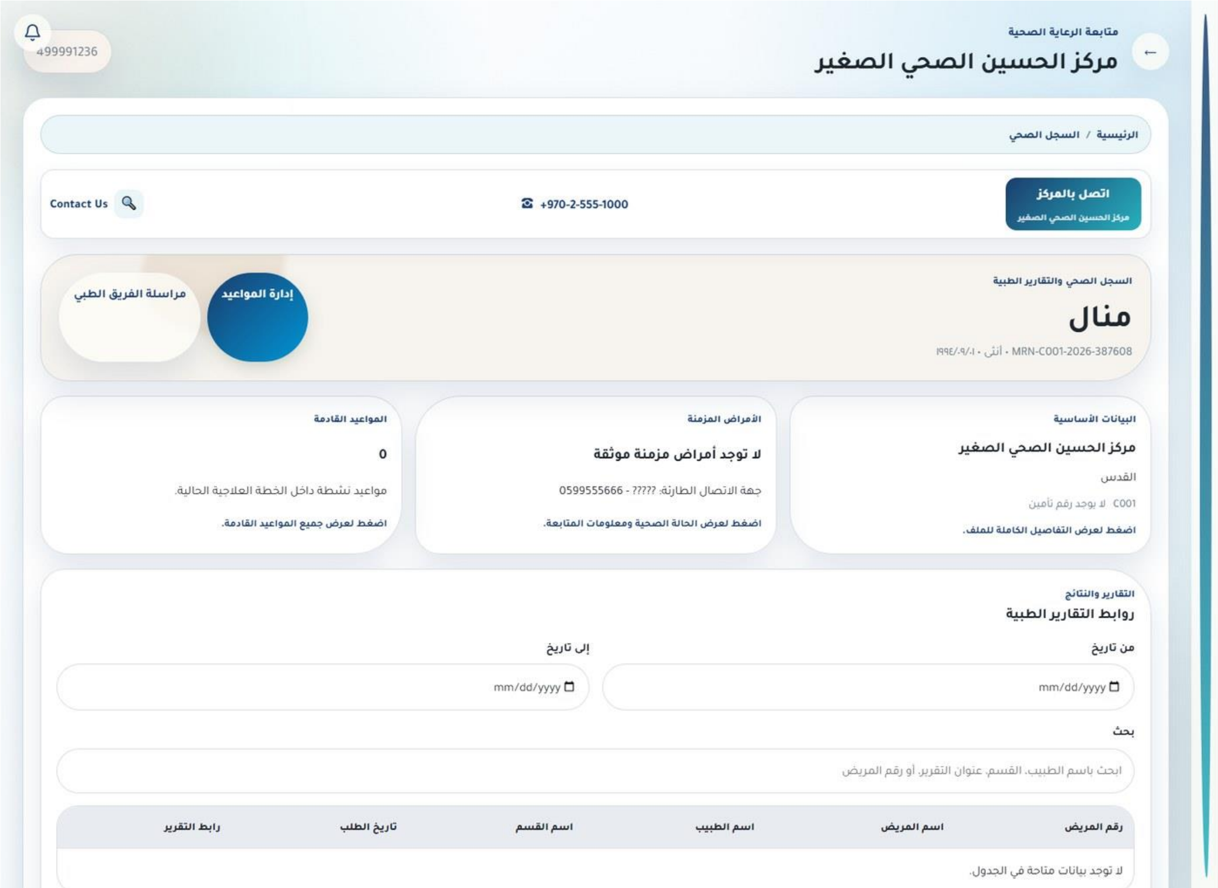

Figure 5.30. Patient medical record — Patient.

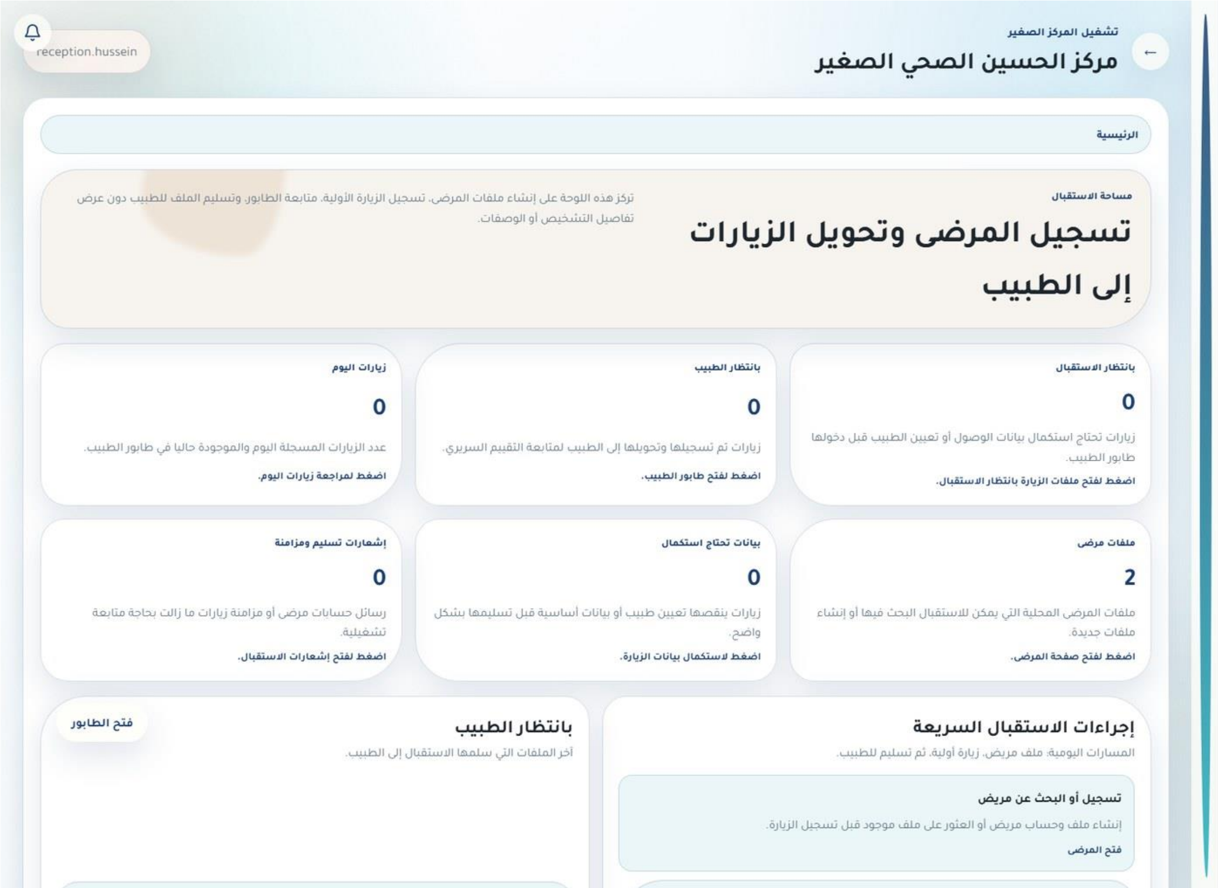

Figure 5.31. Role-aware dashboard — Receptionist.

5.5 Medium-Center Mobile Application

The Expo application connects to the medium-center API and builds its tab/service layout from the authenticated role. Staff roles receive patient, workflow, communication, service, or profile access as appropriate; patients receive appointments, medical record, doctors, messages, permissions, notifications, and care-assistant entry points.

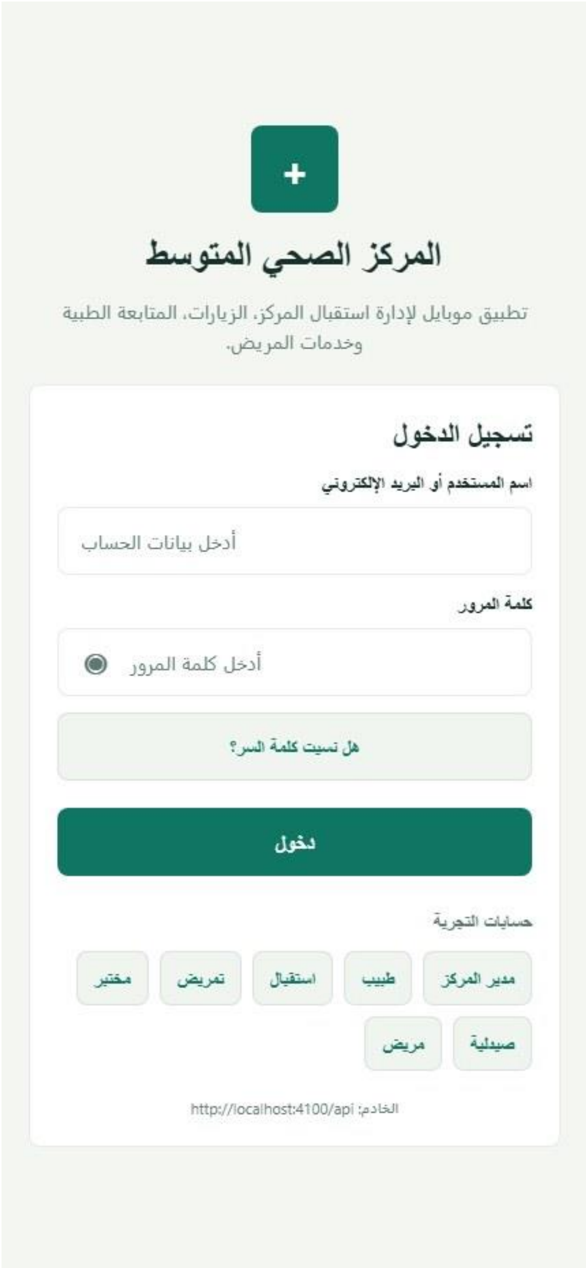

Figure 5.32. Sign-in and account recovery — Unauthenticated user.

Figure 5.33. Secure patient–doctor messaging — Doctor.

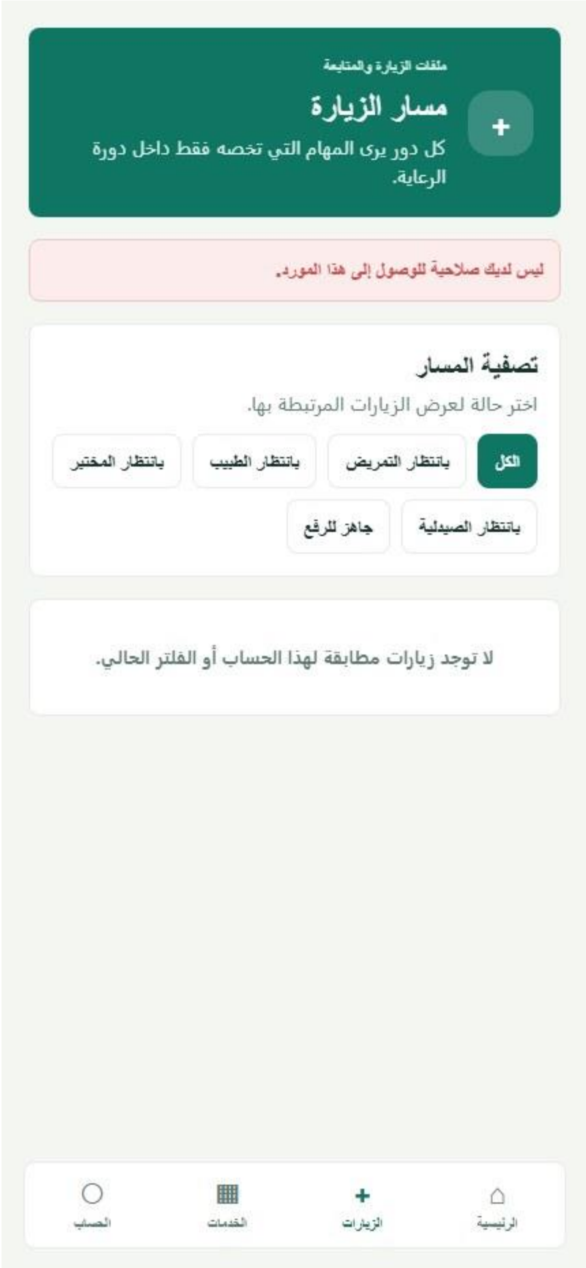

Figure 5.34. Mobile visit workflow — Laboratory staff.

Figure 5.35. Mobile home dashboard — Center manager.

Figure 5.36. Mobile home dashboard — Nurse.

Figure 5.37. Appointments — Patient.

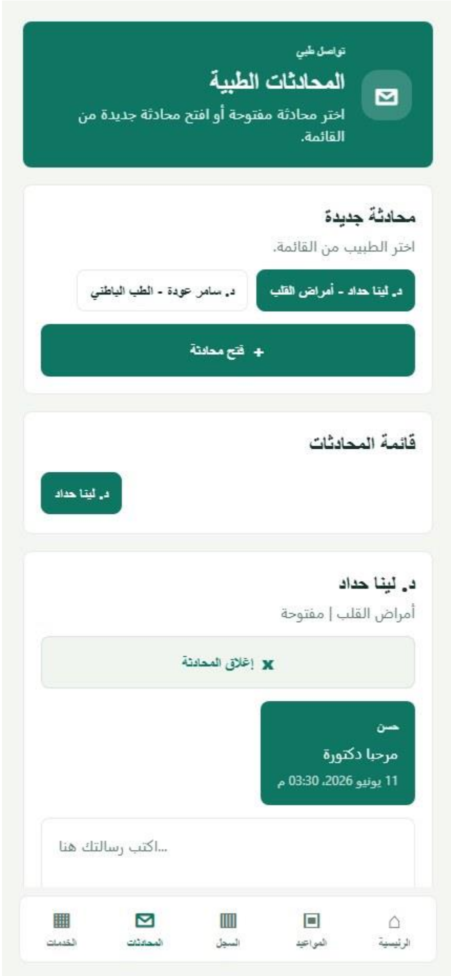

Figure 5.38. Secure patient–doctor messaging — Patient.

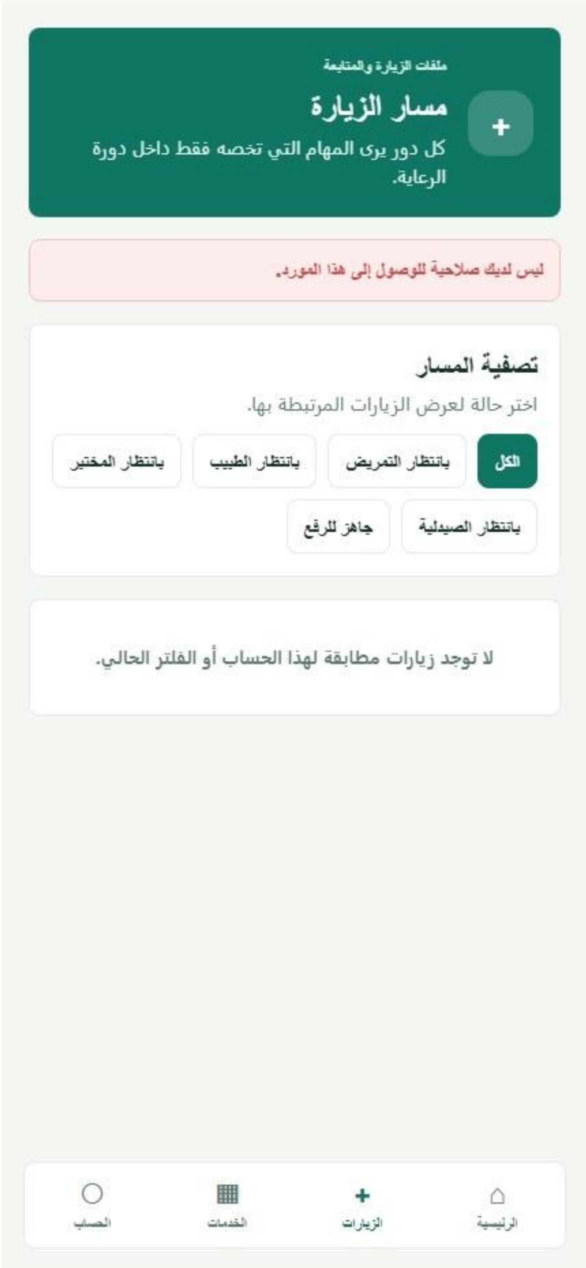

Figure 5.39. Mobile visit workflow — Pharmacist.

Figure 5.40. Mobile home dashboard — Receptionist.

5.6 Implemented Cross-System Behaviors

Table 5.1. Cross-system implementation trace

Behavior	Visible implementation	Server/data support
Authentication	Shared login/reset patterns across clients	JWT middleware and role-aware navigation
Patient identity	Local patient records, unified central records, QR token	Center/patient scoping and audited access
Referral	Sending request → central selection → receiving review → doctor/visit	Explicit status enum and role-specific endpoints
Visit	Reception/triage/doctor/lab/pharmacy/upload as applicable	State transitions and center-type configuration
Communication	Patient–doctor threads, read state, attachment metadata	Authenticated portal routes
Notifications	Local alerts and center–central queue records	Polling processor, attempts, retry, processing log
Audit	Login-sensitive and clinical/administrative event evidence	Actor, entity, metadata, IP/context fields

Chapter 6: Testing and Validation

6.1 Testing Approach

Testing focused on the main workflows that connect users, interfaces, APIs, and stored data. The team used workspace builds, service health checks, role-based login, page-level functional checks, and selected workflow checks. Scenarios that were not completed end to end are reported as partially verified or requiring a final retest.

6.2 Build and Service Checks

Table 6.1. Build and service results

Test area	Method	Status	Observed result
Workspace builds	Compile the API and web workspaces	Passed	The declared build scripts completed successfully.
Central service	Request the service health endpoint	Passed	The central API returned a healthy response.
Medium service	Request the service health endpoint	Passed	The medium-center API returned a healthy response.
Small service	Request the service health endpoint	Passed	The small-center API returned a healthy response.
Web applications	Open the three web clients	Passed	The central, medium, and small interfaces loaded.
Mobile application	Open the Expo client	Passed with warning	The application rendered; an Expo patch-version warning remained.

6.3 Functional Test Matrix

Table 6.2. Functional test matrix

Test scenario	Expected result	Observed result	Status
Central administrator login	Open the central dashboard for an authorized administrator.	Authentication succeeded and the central dashboard opened.	Passed
Medium manager login	Open the manager workspace.	Authentication succeeded and the manager dashboard opened.	Passed
Patient registration and search	Allow reception staff to register and locate a patient.	The receptionist interface and registration/search controls are implemented; a complete submission was not repeated in the final check.	Partially verified
Doctor account and availability	Display center doctors and their available times.	Doctor lists and availability views loaded for authorized roles.	Passed
Appointment creation	Allow a patient to select a doctor and request a valid time.	The appointment page and time-selection flow loaded; final submission was not performed.	Partially verified
Referral creation	Create an inter-center referral request.	The workflow is implemented, but an end-to-end creation was not repeated after the latest database changes.	Final retest required
Receiving-manager decision	Allow the receiving manager to	The decision flow is implemented	Final retest required

Test scenario	Expected result	Observed result	Status
	accept or reject a referral.	but was not completed during the final check.	
Referral doctor assignment	Assign an accepted referral to a doctor.	The assignment flow is implemented but was not completed during the final check.	Final retest required
Visit creation and progression	Create a visit and move it through authorized queue states.	Role dashboards and workflow pages are present; the full sequence requires a database-aligned retest.	Final retest required
Laboratory workflow	Allow laboratory staff to review and process requests.	The laboratory role and mobile workflow opened; result publication was not executed.	Partially verified
Prescription and pharmacy workflow	Review prescriptions and provide pharmacy functions.	Prescription verification, pharmacy records, and the mobile pharmacy workflow opened; dispensing was not executed.	Partially verified
Patient medical record	Allow an authorized patient to view their record.	The small-center patient record and its main information panels loaded.	Passed
Notification handling	Display role-relevant notifications and processing status.	Notification pages loaded in the central and center applications; background delivery was not retested end to end.	Partially verified
Password recovery	Open the recovery form and validate the reset workflow entry point.	The recovery interface opened; external code delivery was not tested.	Partially verified
Role-based access	Direct each account to the functions permitted for its role.	Available roles reached their expected workspaces; systematic negative-authorization testing remains future work.	Partially verified

6.4 Database and Test Limitations

Database migration alignment should be checked against the latest Prisma migrations before final deployment. Earlier local verification recorded schema-version inconsistencies, but these were not re-tested during this editorial revision; therefore, no current defect status is claimed here.

The completed checks do not replace automated unit, integration, end-to-end, security, performance, native-device, or clinical usability testing. The functional matrix reports only what was supported by the available project evidence and avoids treating an implemented interface as proof of every underlying state transition.

Chapter 7: Discussion

7.1 Achievements

Healthcare Ecosystem brings several related applications into one coordinated project. It supports a central administrator, two types of healthcare centers, professional and administrative roles, patient services, and a mobile client. Referral and visit states make responsibilities visible as work moves between users and centers.

The medium-center system demonstrates the most detailed workflow through triage, laboratory, pharmacy, patient permissions, and follow-up functions. The small-center system uses a shorter workflow, showing that the design can adapt to different service capabilities rather than forcing every center to use the same process.

7.2 Architecture and Role Separation

Using TypeScript across the clients and APIs gives the project a consistent implementation language. Prisma models and enums make domain relationships and workflow states explicit. Authentication, role checks, center and patient scoping, input validation, password hashing, and audit records provide an appropriate security foundation for an academic prototype.

Separate APIs and schemas make it easier to customize medium and small centers, but repeated structures also increase maintenance effort. Shared fields, workflow values, and migrations must remain consistent across the services. Gradually moving common validation and domain definitions into shared modules would reduce this risk without removing center-specific behavior.

7.3 Central and Local Responsibilities

The separation between unified network information and local clinical records is one of the main design strengths. The central system coordinates centers, common data, referrals, notifications, reporting, and audit views. Local systems retain the detailed visits, prescriptions, laboratory work, appointments, and patient interactions needed for everyday operation.

Inter-center referrals depend on both sides of this design. Local staff create and act on the referral, while the central layer helps identify a suitable receiving center and carries status updates. Notification and audit records improve traceability, although the current polling approach may need further optimization as the number of centers grows.

7.4 Project Limitations

- The project was developed and tested in a local environment rather than a deployed healthcare network.
- Automated unit, integration, end-to-end, performance, and security tests are not yet comprehensive.
- Native-device behavior, push notifications, camera-based QR use, and offline conditions require broader testing.
- Formal usability studies with healthcare professionals and patients have not yet been completed.
- Database migration alignment should be confirmed again before final deployment.
- The current APIs use project-specific JSON structures rather than HL7 FHIR resources.

Healthcare Ecosystem is an academic prototype and is not intended to replace professional medical judgment. Any future use with real patient data would require clinical review, privacy and security assessment, and approval under the applicable institutional rules.

Chapter 8: Conclusion and Future Work

8.1 Conclusions

Healthcare Ecosystem is an academic implementation of a multi-center healthcare information platform. It brings together central administration, medium- and small-center workflows, a role-based mobile application, patient and clinician records, appointments, visits, referrals, laboratory and pharmacy activities, messaging, notifications, prescription verification, and audit records. The design shows how several professional roles can share information while each center keeps its own operational workflow.

The project meets its main goal of demonstrating coordinated healthcare work across several applications and user roles. Its architecture provides a practical base for continued development, while the remaining testing, interoperability, and deployment work is appropriate for future extension of a graduation-project prototype.

8.2 Future Work

- Add automated unit, API integration, and end-to-end tests for the main workflows and role restrictions.
- Strengthen production security, deployment configuration, backup procedures, and migration checks.
- Introduce HL7 FHIR mappings for controlled exchange with external healthcare systems.
- Test native mobile builds, secure device storage, camera-based QR scanning, and push notifications.
- Improve notification delivery reliability and monitoring between local and central systems.
- Conduct usability studies with healthcare professionals and patients, then refine the interfaces from their feedback.
- Reduce duplicated validation, domain definitions, and shared workflow code gradually across the services.

8.3 Final Statement

Healthcare Ecosystem demonstrates the practical application of full-stack engineering to a complex, role-based healthcare domain. The project provides a clear foundation for further testing, improvement, and responsible deployment research.

References

[1] World Health Organization, Global Strategy on Digital Health 2020–2027. Geneva: WHO, 2025. https://www.who.int/publications/i/item/9789240116870 (accessed 29 August 2026).

[2] Health Level Seven International, FHIR Release 5: Overview. https://hl7.org/fhir/ (accessed 29 August 2026).

[3] W. Jansen, A Revised Model for Role Based Access Control, NISTIR 6192. National Institute of Standards and Technology, 1998. https://doi.org/10.6028/NIST.IR.6192.

[4] OWASP Foundation, OWASP API Security Top 10 — 2023. https://owasp.org/API-Security/editions/2023/en/0x00-header/ (accessed 29 August 2026).

[5] N. Menachemi and T. H. Collum, "Benefits and drawbacks of electronic health record systems," Risk Management and Healthcare Policy, vol. 4, pp. 47–55, 2011. PMID: 22312227.

[6] R. S. Rudin, A. Motala, C. L. Goldzweig, and P. G. Shekelle, "Usage and effect of health information exchange: a systematic review," Annals of Internal Medicine, vol. 161, no. 11, pp. 803–811, 2014. https://doi.org/10.7326/M14-0877.

[7] Meta Platforms, Inc., React Documentation. https://react.dev/ (accessed 29 August 2026).

[8] Microsoft, TypeScript Documentation. https://www.typescriptlang.org/docs/ (accessed 29 August 2026).

[9] OpenJS Foundation, Node.js Documentation. https://nodejs.org/en/learn (accessed 29 August 2026).

[10] PostgreSQL Global Development Group, PostgreSQL Documentation. https://www.postgresql.org/docs/current/ (accessed 29 August 2026).

[11] Prisma Data, Inc., Prisma ORM Documentation (schema and Prisma Migrate). https://www.prisma.io/docs/orm (accessed 29 August 2026).

[12] S. M. Alawneh and Y. A. Saad, UniServe — Software Graduation Project Report, Computer Engineering / Software Collection, An-Najah National University, 2026. https://repository.najah.edu/items/d687e742-4202-4378-89e7-1d3402f9286c.

[13] A. Tubaila and A. Abuzahra, MARS — Graduation Project Report (Software), Computer Engineering / Software Collection, An-Najah National University, 2026. https://repository.najah.edu/items/bb23b772-616b-4219-8a64-c82037584c5d.

[14] An-Najah National University, "History and Traditions — An-Najah University Logo." https://www.najah.edu/en/about/history-and-traditions/ (accessed 29 August 2026).

[15] Expo, Expo Documentation. https://docs.expo.dev/ (accessed 29 August 2026).

[16] Express.js, Express Web Framework Documentation. https://expressjs.com/ (accessed 29 August 2026).

Appendix B: Technical Summary

This appendix summarizes the main packages, database entities, API groups, and technology versions used in Healthcare Ecosystem.

Table B.1. System and package summary

Package group	Responsibility
Central API / Web	Network administration, centers, shared data, reports, notifications, and audit views
Medium API / Web	Full center workflow for managers, clinical staff, support staff, and patients
Small API / Web	Reduced center workflow for managers, doctors, receptionists, and patients
Mobile	Role-aware medium-center access for staff and patients
Shared package	Common TypeScript domain declarations

Table B.2. Key database entities

Entity	Purpose
UnifiedPatient / LocalPatient	Connect a network patient identity with the record used by an individual center.
LocalVisit	Stores a center visit and its workflow state.
CentralReferral	Coordinates an inter-center referral and its decisions.
CenterUserAccount / CenterDoctorProfile	Represent center users, roles, and doctor information.
LocalPrescription	Stores medication instructions associated with local care.
LabRequestLocal	Represents a laboratory request and result workflow.
IncomingNotification / OutgoingNotification	Carry updates between local and central services.
AuditLog	Records important actions and their context.

Table B.3. Major API groups

API group	Main functions
Authentication	Login, current account, and password recovery
Central administration	Centers, master data, analytics, reports, and audit records
Patients and doctors	Registration, profiles, QR lookup, doctor records, and availability
Appointments and visits	Scheduling, intake, queues, treatment stages, and completion
Referrals	Creation, receiving-center decision, assignment, and status updates
Laboratory and pharmacy	Requests, results, prescriptions, verification, dispensing, and inventory
Communication	Messages, notifications, and processing status
Patient portal	Appointments, medical record, doctors, messages, and permissions

Table B.4. Selected technology versions

Area	Technology/version
Web	React 18.3; React Router 6.28; Vite 5.4
Mobile	React 19.1; React Native 0.81.5; Expo 54
Language	TypeScript 5.6
Server	Node.js; Express 4.19
Database	PostgreSQL; Prisma 5.15
Security and validation	jsonwebtoken 9; bcryptjs 2.4; Zod 3.23; Helmet 7.1